



User Guide

EventBridge Scheduler



EventBridge Scheduler: User Guide

Copyright © 2025 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

What is EventBridge Scheduler?	1
Key features of EventBridge Scheduler	1
Accessing EventBridge Scheduler	2
Setting up	3
Sign up for AWS	3
Create an IAM user	3
Use managed policies	4
Set up the execution role	5
Set up a target	9
What's next?	11
Getting started	12
Prerequisites	12
Using the console	13
Using the AWS CLI	16
Using the SDKs	17
What's next?	18
Schedule types	19
Rate-based schedules	19
Syntax	20
Examples	20
Cron-based schedules	21
Syntax	21
Examples	22
One-time schedules	23
Syntax	23
Examples	23
Time zones	24
Daylight savings time	24
Managing a schedule	26
Changing the schedule state	27
Configuring flexible time windows	28
Configuring a DLQ	29
Create an Amazon SQS queue	30
Set up execution role permissions	31

Specify a dead-letter queue	31
Retrieve the dead-letter event	33
Deleting a schedule	35
Deletion after schedule completion	36
Manual deletion	37
What's next?	37
Managing a schedule group	38
Creating a schedule group	39
Step one: Create a new schedule group	39
Associating a schedule	40
Deleting a schedule group	42
Related resources	43
Managing targets	45
Using templated targets	45
Amazon SQS SendMessage	47
Lambda Invoke	49
Step Functions StartExecution	51
Using universal targets	53
Unsupported actions	53
Examples	54
Adding context attributes	56
What's next?	57
AWS PrivateLink	58
Considerations	58
Create an interface endpoint	58
Create an endpoint policy	59
Security	60
Managing access	60
Audience	61
Authenticating with identities	62
Managing access using policies	65
Integration with IAM	67
Using identity-based policies	74
Confused deputy prevention	84
Troubleshooting	86
Data protection	88

Encryption at rest	89
Encryption in transit	97
Compliance validation	97
Resilience	98
Infrastructure Security	99
Monitoring and metrics	100
Monitoring with CloudWatch	100
Terms	101
Dimensions	101
Accessing metrics	102
List of metrics	102
Usage metrics	107
Monitoring with CloudTrail logs	109
EventBridge Scheduler information in CloudTrail	110
Understanding EventBridge Scheduler log file entries	111
Quotas	112
Troubleshooting quotas	121
ServiceQuotaExceededException	122
Document history	124

What is Amazon EventBridge Scheduler?

Amazon EventBridge Scheduler is a serverless scheduler that allows you to create, run, and manage tasks from one central, managed service. Highly scalable, EventBridge Scheduler allows you to schedule millions of tasks that can invoke more than 270 AWS services and over 6,000 API operations. Without the need to provision and manage infrastructure, or integrate with multiple services, EventBridge Scheduler provides you with the ability to deliver schedules at scale and reduce maintenance costs.

EventBridge Scheduler delivers your tasks reliably, with built-in mechanisms that adjust your schedules based on the availability of downstream targets. With EventBridge Scheduler, you can create schedules using cron and rate expressions for recurring patterns, or configure one-time invocations. You can set up flexible time windows for delivery, define retry limits, and set the maximum retention time for failed triggers.

Topics

- [Key features of EventBridge Scheduler](#)
- [Accessing EventBridge Scheduler](#)

Key features of EventBridge Scheduler

EventBridge Scheduler offers the following key features that you can use to configure targets and scale your schedules.

- **Templated targets** – EventBridge Scheduler supports templated targets to perform common API operations using Amazon SQS, Amazon SNS, Lambda, and EventBridge. With predefined targets, you can configure your schedules quickly using the EventBridge Scheduler console, the EventBridge Scheduler SDK, or the AWS CLI.
- **Universal targets** – EventBridge Scheduler provides a universal target parameter (UTP) that you can use to create customized triggers that target more than 270 AWS services and over 6,000 API operations on a schedule. With UTP, you can configure your customized triggers using the EventBridge Scheduler console, the EventBridge Scheduler SDK, or the AWS CLI.
- **Flexible time windows** – EventBridge Scheduler supports flexible time windows, allowing you to disperse your schedules and improve the reliability of your triggers for use cases that do not require precise scheduled invocation of targets.

- **Retries** – EventBridge Scheduler provides at-least-once event delivery to targets, meaning that at least one delivery succeeds with a response from the target. EventBridge Scheduler allows you to set the number of retries for your schedule for a failed task. EventBridge Scheduler retries failed tasks with delayed attempts to improve the reliability of your schedule and ensure targets are available.

Accessing EventBridge Scheduler

You can use EventBridge Scheduler via the EventBridge console, the EventBridge Scheduler SDK, the AWS CLI, or by directly using the EventBridge Scheduler API.

Setting up Amazon EventBridge Scheduler

Before you can use EventBridge Scheduler, you must complete the following steps.

Topics

- [Sign up for AWS](#)
- [Create an IAM user](#)
- [Use managed policies](#)
- [Set up the execution role](#)
- [Set up a target](#)
- [What's next?](#)

Sign up for AWS

If you do not have an AWS account, complete the following steps to create one.

To sign up for an AWS account

1. Open <https://portal.aws.amazon.com/billing/signup>.
2. Follow the online instructions.

Part of the sign-up procedure involves receiving a phone call and entering a verification code on the phone keypad.

When you sign up for an AWS account, an *AWS account root user* is created. The root user has access to all AWS services and resources in the account. As a security best practice, assign administrative access to a user, and use only the root user to perform [tasks that require root user access](#).

Create an IAM user

To create an administrator user, choose one of the following options.

Choose one way to manage your administrator	To	By	You can also
In IAM Identity Center (Recommended)	Use short-term credentials to access AWS. This aligns with the security best practices . For information about best practices , see Security best practices in IAM in the <i>IAM User Guide</i> .	Following the instructions in Getting started in the <i>AWS IAM Identity Center User Guide</i> .	Configure programmatic access by Configuring the AWS CLI to use AWS IAM Identity Center in the <i>AWS Command Line Interface User Guide</i> .
In IAM (Not recommended)	Use long-term credentials to access AWS.	Following the instructions in Create an IAM user for emergency access in the <i>IAM User Guide</i> .	Configure programmatic access by Manage access keys for IAM users in the <i>IAM User Guide</i> .

Use managed policies

In the previous step, you set up an IAM user with the credentials to access your AWS resources. In most cases, to use EventBridge Scheduler securely, we recommend that you create separate users, groups, or roles with only the necessary permissions to use EventBridge Scheduler. EventBridge Scheduler supports the following managed policies for common use cases.

- [the section called “AmazonEventBridgeSchedulerFullAccess”](#) – Grants full access to EventBridge Scheduler using the console and the API.
- [the section called “AmazonEventBridgeSchedulerReadOnlyAccess”](#) – Grants read-only access to EventBridge Scheduler.

You can attach these managed policies to your IAM principals the same way you attached the `AdministratorAccess` policy in the previous step. For more information about managing access to EventBridge Scheduler using identity-based IAM policies, see [the section called “Using identity-based policies”](#).

Set up the execution role

An *execution role* is an IAM role that EventBridge Scheduler assumes in order to interact with other AWS services on your behalf. You attach permission policies to this role to grant EventBridge Scheduler access to invoke targets.

You can also create a new execution role when you use the console to [create a new schedule](#). If you use the console, EventBridge Scheduler creates a role on your behalf with permissions based on the target you choose. When EventBridge Scheduler creates a role for you, the role's trust policy includes [condition keys](#) that limit which principals can assume the role on your behalf. This guards against the potential [confused deputy security issue](#).

The following steps describe how to create a new execution role and how to grant EventBridge Scheduler access to invoke a target. This topic describes permissions for popular templated targets. For information on adding permissions for other targets, see [the section called “Using templated targets”](#).

To create an execution role using the AWS CLI

1. Copy the following assume role JSON policy and save it locally as `Scheduler-Execution-Role.json`. This trust policy allows EventBridge Scheduler to assume the role on your behalf.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "scheduler.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

⚠ Important

To set up an execution role in a production environment, we recommend implementing additional safeguards for preventing confused deputy issues. For more information and an example policy, see [the section called “Confused deputy prevention”](#).

2. From the AWS Command Line Interface (AWS CLI), enter the following command to create a new role. Replace *SchedulerExecutionRole* with the name you want to give this role.

```
$ aws iam create-role --role-name SchedulerExecutionRole --assume-role-policy-document file://Scheduler-Execution-Role.json
```

If successful, you'll see the following output:

```
{
  "Role": {
    "Path": "/",
    "RoleName": "Scheduler-Execution-Role",
    "RoleId": "BR1L2DZK3K4CTL5ZF9EIL",
    "Arn": "arn:aws:iam::123456789012:role/SchedulerExecutionRole",
    "CreateDate": "2022-03-10T18:45:01+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": "scheduler.amazonaws.com"
          },
          "Action": "sts:AssumeRole"
        }
      ]
    }
  }
}
```

3. To create a new policy that allows EventBridge Scheduler to invoke a target, choose one of the following common targets. Copy the JSON permission policy and save it locally as a `.json` file.

Amazon SQS – SendMessage

The following allows EventBridge Scheduler to call the `sqs:SendMessage` action on all Amazon SQS queues in your account.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sqs:SendMessage"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Amazon SNS – Publish

The following allows EventBridge Scheduler to call the `sns:Publish` action on all Amazon SNS topics in your account.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sns:Publish"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Lambda – Invoke

The following allows EventBridge Scheduler to call the `lambda:InvokeFunction` action on all Lambda functions in your account.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "lambda:InvokeFunction"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

4. Run the following command to create the new permission policy. Replace *PolicyName* with the name you want to give this policy.

```
$ aws iam create-policy --policy-name PolicyName --policy-document file://
PermissionPolicy.json
```

If successful, you'll see the following output. Note the policy ARN. You use this ARN in the next step to attach the policy to our execution role.

```
{
  "Policy": {
    "PolicyName": "PolicyName",
    "CreateDate": "2022-03-01T19:31:18.620Z",
    "AttachmentCount": 0,
    "IsAttachable": true,
    "PolicyId": "ZXR6A36LTYANPAI7NJ5UV",
    "DefaultVersionId": "v1",
    "Path": "/",
    "Arn": "arn:aws:iam::123456789012:policy/PolicyName",
    "UpdateDate": "2022-03-01T19:31:18.620Z"
  }
}
```

5. Run the following command to attach the policy to your execution role. Replace *your-policy-arn* with the ARN of the policy you created in the previous step. Replace *SchedulerExecutionRole* with the name of your execution role.

```
$ aws iam attach-role-policy --policy-arn your-policy-arn --role-name SchedulerExecutionRole
```

The `attach-role-policy` operation doesn't return a response on the command line.

Set up a target

Before you create an EventBridge Scheduler schedule, you need at least one target for your schedule to invoke. You can use an existing AWS resource, or create a new one. The following steps show how to create a new standard Amazon SQS queue with AWS CloudFormation.

To create a new Amazon SQS queue

1. Copy the following JSON AWS CloudFormation template and save it locally as `SchedulerTargetSQS.json`.

```
{
  "AWSTemplateFormatVersion": "2010-09-09",
  "Resources": {
    "MyQueue": {
      "Type": "AWS::SQS::Queue",
      "Properties": {
        "QueueName": "MyQueue"
      }
    }
  },
  "Outputs": {
    "QueueName": {
      "Description": "The name of the queue",
      "Value": {
        "Fn::GetAtt": [
          "MyQueue",
          "QueueName"
        ]
      }
    },
    "QueueURL": {
      "Description": "The URL of the queue",
      "Value": {
        "Ref": "MyQueue"
      }
    }
  }
}
```

```

    }
  },
  "QueueARN": {
    "Description": "The ARN of the queue",
    "Value": {
      "Fn::GetAtt": [
        "MyQueue",
        "Arn"
      ]
    }
  }
}
}
}

```

2. From the AWS CLI, run the following command to create an AWS CloudFormation stack from the Scheduler-Target-SQS.json template.

```
$ aws cloudformation create-stack --stack-name Scheduler-Target-SQS --template-body
file://Scheduler-Target-SQS.json
```

If successful, you'll see the following output:

```
{
  "StackId": "arn:aws:cloudformation:us-west-2:123456789012:stack/Scheduler-
Target-SQS/1d2af345-a121-12eb-abc1-012e34567890"
}
```

3. Run the following command to view summary information for your AWS CloudFormation stack. This information includes the status of the stack and the outputs specified in the template.

```
$ aws cloudformation describe-stacks --stack-name Scheduler-Target-SQS
```

If successful, the command creates the Amazon SQS queue and returns the following output:

```
{
  "Stacks": [
    {
      "StackId": "arn:aws:cloudformation:us-west-2:123456789012:stack/
Scheduler-Target-SQS/1d2af345-a121-12eb-abc1-012e34567890",
      "StackName": "Scheduler-Target-SQS",

```

```

    "CreationTime": "2022-03-17T16:21:29.442000+00:00",
    "RollbackConfiguration": {},
    "StackStatus": "CREATE_COMPLETE",
    "DisableRollback": false,
    "NotificationARNs": [],
    "Outputs": [
      {
        "OutputKey": "QueueName",
        "OutputValue": "MyQueue",
        "Description": "The name of the queue"
      },
      {
        "OutputKey": "QueueARN",
        "OutputValue": "arn:aws:sqs:us-west-2:123456789012:MyQueue",
        "Description": "The ARN of the queue"
      },
      {
        "OutputKey": "QueueURL",
        "OutputValue": "https://sqs.us-west-2.amazonaws.com/123456789012/MyQueue",
        "Description": "The URL of the queue"
      }
    ],
    "Tags": [],
    "EnableTerminationProtection": false,
    "DriftInformation": {
      "StackDriftStatus": "NOT_CHECKED"
    }
  }
]
}

```

Later in this guide, you'll use the value for QueueARN to set up the queue as a target for EventBridge Scheduler.

What's next?

After you've completed the set up step, use the [Getting started](#) guide to create your first EventBridge Scheduler scheduler and invoke a target.

Getting started with EventBridge Scheduler

This topic describes creating a new EventBridge Scheduler schedule. You use the EventBridge Scheduler console, AWS Command Line Interface (AWS CLI), or AWS SDKs to create a schedule with a templated Amazon SQS target. Then, you'll set up logging, configure retries, and set a maximum retention time for failed tasks. After you create the schedule, you'll verify that your schedule successfully invokes the target and sends a message to the target queue.

Note

To follow this guide, we recommend that you set up IAM users with the minimum required permissions described in [the section called “Using identity-based policies”](#). After you create and configure a user, run the following command to set your access credentials. You'll need your access key ID and secret access key to configure the AWS CLI.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

For more information about different ways you can set your credentials, see [Configuration settings and precedence](#) in the *AWS Command Line Interface User Guide for Version 2*.

Topics

- [Prerequisites](#)
- [Create a schedule using the EventBridge Scheduler console](#)
- [Create a schedule using the AWS CLI](#)
- [Create a schedule using the EventBridge Scheduler SDKs](#)
- [What's next?](#)

Prerequisites

Before attempting the steps in this section, you must do the following:

- Complete the tasks described in [Setting up](#)

Create a schedule using the EventBridge Scheduler console

To create a new schedule using the console

1. Sign in to the AWS Management Console, then choose the following link to open the EventBridge Scheduler section of the EventBridge console: <https://us-west-2.console.aws.amazon.com/scheduler/home?region=us-west-2#home>

Note

You can switch your AWS Region by using the AWS Management Console's Region selector.

2. On the **Schedules** page, choose **Create schedule**.
3. On the **Specify schedule detail** page, in the **Schedule name and description** section, do the following:
 - a. For **Schedule name**, enter a name for your schedule. For example, **MyTestSchedule**
 - b. For **Description - optional**, enter a description for your schedule. For example, **My first schedule**.
 - c. For **Schedule group**, choose a schedule group from the drop down options. If you haven't previously made any schedule groups, you can choose the default group for your schedule. To create a new schedule group, choose the **create your own schedule** link in the console description. You use schedule groups to add tags to groups of schedules.
4. In the **Schedule pattern** section, do the following:
 - a. For **Occurrence**, choose one of the following pattern options. The configuration options change depending on which pattern that you select.
 - **One-time schedule** – A one-time schedule invokes a target only once at the date and time that you specify.

For **Date and time**, enter a valid date in YYYY/MM/DD format. Then, specify a timestamp in 24-hour hh:mm format. Finally, choose a timezone from the drop down options.

- **Recurring schedule** – A recurring schedule invokes a target at a rate that you specify using a **cron** expression or rate expression.

Choose **Cron-based schedule** to configure a schedule by using a **cron** expression. To use a rate expression, choose **Rate-based schedule** and enter a positive number for **Value**, then choose a **Unit** from the drop down options.

For more information on using cron and rate expressions, see [Schedule types](#).

- b. For **Flexible time window**, choose **Off** to turn off the option, or choose one of the pre-defined time windows from the drop down list. For example, if you choose **15 minutes** and you set a recurring schedule to invoke its target once every hour, the schedule runs within 15 minutes after the start of every hour.
5. If you chose **Recurring schedule** in the previous step, in the **Timeframe** section, specify a timezone, and optionally set a start date and time, and an end date and time for the schedule. A recurring schedule without a start date will begin as soon as it is created and available. A recurring schedules without an end date will continue to invoke its target indefinitely.
6. Choose **Next**.
7. On the **Select target** page, do the following:
 - a. Select **Templated targets** and choose a target API. For this example, we'll choose the **Amazon SQS SendMessage** templated target.
 - b. On the **SendMessage** section, for **SQS queue**, choose an existing Amazon SQS queue ARN such as `arn:aws:sqs:us-west-2:123456789012:TestQueue` from the drop down list. To create a new queue, choose **Create new SQS queue** to navigate to the Amazon SQS console. After finish creating a queue, return to the EventBridge Scheduler console and refresh the drop down. Your new queue ARN appears and can be selected.
 - c. For **Target**, enter the payload you want EventBridge Scheduler to deliver to the target. For this example, we'll send the following message to the target queue: **Hello, it's EventBridge Scheduler.**
8. Choose **Next**, then on the **Settings - optional** page, do the following:
9.
 - a. In the **Schedule state** section, for **Enable schedule**, toggle feature on or off using the switch. By default, the EventBridge Scheduler enables your schedule.
 - b. In the **Action after schedule completion** section, configure the action EventBridge Scheduler takes after the schedule completes:

- Choose **DELETE** if you want the schedule to be automatically deleted. For one-time schedules, this occurs after the schedule invokes the target once. For recurring schedules, this occurs after the schedule's last planned invocation. For more information about automatic deletion, see [the section called “Deletion after schedule completion”](#).
 - Choose **NONE**, or do not choose a value, if you do not want EventBridge Scheduler to take any action after the schedule completes.
- c. In the **Retry policy and dead-letter queue (DLQ)** section, for **Retry policy**, turn **Retry** on to configure a retry policy for your schedule. With retry policies, if a schedule fails to invoke its target, EventBridge Scheduler re-runs the schedule. If configured, you must set the maximum retention time and retries for the schedule.
- d. For **Maximum age of event - *optional***, enter the maximum **hour(s)** and **min(s)** that EventBridge Scheduler must keep an unprocessed event.

 Note

The maximum value is 24 hours.

- e. For **Maximum retries**, enter the maximum number of times EventBridge Scheduler retries the schedule if the target returns an error.

 Note

The maximum value is 185 retries.

- f. For **Dead-letter queue (DLQ)**, choose from the following options:
- **None** – Choose this option if you do not want to configure a DLQ.
 - **Select an Amazon SQS queue in my AWS account as DLQ** – Choose this option, then select a queue ARN from the drop down list, configure a DLQ the same AWS account as the one where you're creating the schedule.
 - **Specify an Amazon SQS queue in other AWS account as DLQ** – Choose this option, then enter the ARN of the queue configure as the DLQ, if the queue is in another AWS account. You must enter the exact ARN for the queue in order to use this option.
- g. In the **Encryption** section, choose **Customize encryption settings (advanced)** to use a customer managed KMS key to encrypt your target input. If you choose this option, enter an existing KMS key ARN or choose **Create an AWS KMS key** to navigate to the AWS KMS

console. For more information on how EventBridge Scheduler encrypts your data at rest, see [the section called “Encryption at rest”](#).

- h. For **Permissions**, choose **Use existing role**, then select the role you created during the [setup](#) procedure from the drop down list. You can also choose **Go to IAM console** to create a new role.

If you would like EventBridge Scheduler to create a new execution role for you, choose **Create new role for this schedule** instead. Then, enter a name for **Role name**. If you choose this option, EventBridge Scheduler adds the required permissions necessary for your templated target to the role.

10. Choose **Next**.
11. In the **Review and create schedule** page, review the details of your schedule. In each section, choose **Edit** to go back to that step and edit its details.
12. Choose **Create schedule** to finish creating your new schedule. You can view a list of your new and existing schedules on the **Schedules** page. Under the **Status** column, verify that your new schedule is **Enabled**.
13. To verify that your schedule invokes the Amazon SQS target, open the Amazon SQS console and do the following:
 - a. Choose the target queue from the **Queues** list.
 - b. Choose **Send and receive messages**.
 - c. On the **Send and receive messages** page, under **Receive messages**, choose **Poll for messages** to retrieve the test messages your schedule sent to the target queue.

Create a schedule using the AWS CLI

The following example shows how to use the AWS CLI command [create-schedule](#) to create a EventBridge Scheduler schedule with a templated Amazon SQS target. Replace the placeholder values for the following parameters with your information:

- **--name** – Enter a name for the schedule.
- **RoleArn** – Enter the ARN for the execution role you want to associate with the schedule.
- **Arn** – Enter the ARN for the target. In this case, the target is an Amazon SQS queue.
- **Input** – Enter a message that EventBridge Scheduler delivers to the target queue.

```
$ aws scheduler create-schedule --name sqs-templated-schedule --schedule-expression  
'rate(5 minutes)' \  
--target '{"RoleArn": "ROLE_ARN", "Arn": "QUEUE_ARN", "Input": "TEST_PAYLOAD" }' \  
--flexible-time-window '{ "Mode": "OFF" }'
```

Create a schedule using the EventBridge Scheduler SDKs

In the following example, you use the EventBridge Scheduler SDKs to create a EventBridge Scheduler schedule with a templated Amazon SQS target.

Example Python SDK

```
import boto3  
scheduler = boto3.client('scheduler')  
  
flex_window = { "Mode": "OFF" }  
  
sqs_templated = {  
    "RoleArn": "<ROLE_ARN>",  
    "Arn": "<QUEUE_ARN>",  
    "Input": "Message for scheduleArn: '<aws.scheduler.schedule-arn>', scheduledTime:  
'<aws.scheduler.scheduled-time>'"  
}  
  
scheduler.create_schedule(  
    Name="sqs-python-templated",  
    ScheduleExpression="rate(5 minutes)",  
    Target=sqs_templated,  
    FlexibleTimeWindow=flex_window)
```

Example Java SDK

```
package com.example;  
  
import software.amazon.awssdk.regions.Region;  
import software.amazon.awssdk.services.scheduler.SchedulerClient;  
import software.amazon.awssdk.services.scheduler.model.*;  
  
public class MySchedulerApp {
```

```
public static void main(String[] args) {

    final SchedulerClient client = SchedulerClient.builder()
        .region(Region.US_WEST_2)
        .build();

    Target sqsTarget = Target.builder()
        .roleArn("<ROLE_ARN>")
        .arn("<QUEUE_ARN>")
        .input("Message for scheduleArn: '<aws.scheduler.schedule-arn>',
scheduledTime: '<aws.scheduler.scheduled-time>'")
        .build();

    CreateScheduleRequest createScheduleRequest = CreateScheduleRequest.builder()
        .name("<SCHEDULE_NAME>")
        .scheduleExpression("rate(10 minutes)")
        .target(sqsTarget)
        .flexibleTimeWindow(FlexibleTimeWindow.builder()
            .mode(FlexibleTimeWindowMode.OFF)
            .build())
        .build();

    client.createSchedule(createScheduleRequest);
    System.out.println("Created schedule with rate expression and an Amazon SQS
templated target");
}
}
```

What's next?

- For more information about managing your schedule using the console, AWS CLI, or the EventBridge Scheduler SDK, see [Managing a schedule](#).
- For more information about how to configure templated targets and learn about using the universal target parameter, see [Managing targets](#).
- For more information about the EventBridge Scheduler data types and API operations, see the [EventBridge Scheduler API Reference](#).

Schedule types in EventBridge Scheduler

The following topic describes the different schedule types that Amazon EventBridge Scheduler supports, as well as how EventBridge Scheduler handles daylight savings time, and scheduling in different time zones. You can choose from three schedule types when configuring your schedule: *rate-based*, *cron-based*, and *one-time* schedules.

Both rate-based and cron-based schedules are recurring schedules. You configure each recurring schedule type using a *schedule expression* for the type of schedule you want to configure, and specifying a time zone in which EventBridge Scheduler evaluates the expression.

A one-time schedule is a schedule that invokes a target only once. You configure a one-time schedule when by specifying the time, date, and time zone in which EventBridge Scheduler evaluates the schedule.

Note

All schedule types on EventBridge Scheduler invoke their targets with 60 second precision. This means that if you set your schedule to run at 1:00, it will invoke the target API between 1:00:00 and 1:00:59, assuming that a flexible time window is not set.

Use the following sections to learn about configuring schedule expressions for each recurring schedule type, and how to set up a one-time schedule on EventBridge Scheduler.

Topics

- [Rate-based schedules](#)
- [Cron-based schedules](#)
- [One-time schedules](#)
- [Time zones on EventBridge Scheduler](#)
- [Daylight savings time on EventBridge Scheduler](#)

Rate-based schedules

A rate-based schedule starts after the start date you specify for your schedule, and runs at a regular rate that you define until the schedule's end date. You can set up most common recurrent

scheduling use cases using a rate-based schedule. For example, if you want a schedule that invokes its target every 15 minutes, once every two hours, or once every five days, you can use a rate-based schedule to achieve this. You configure a rate-based schedule using a *rate expression*.

With rate-based schedules, you use the [StartDate](#) property to set the first occurrence of the schedule. If you do not provide a StartDate for a rate-based schedule, your schedule starts invoking the target immediately.

Rate expressions have two required fields separated by a white space, as shown in the following.

Syntax

```
rate(value unit)
```

value

A positive number.

unit

The unit of time you want your schedule to invoke its target.

Valid inputs: minutes | hours | days

Examples

The following example shows how to use rate expressions with the AWS CLI `create-schedule` command to configure a rate-based schedule. This example creates a schedule that runs every five minutes and delivers a message to an Amazon SQS queue, using the templated `SqsParameters` target type.

Because this example does not set a value for the `--start-date` parameter, the schedule starts invoking its target immediately after you create and activate it.

```
$ aws scheduler create-schedule --schedule-expression 'rate(5 minutes)' --  
name schedule-name \  
--target '{"RoleArn": "role-arn", "Arn": "QUEUE_ARN", "Input": "TEST_PAYLOAD" }' \  
--flexible-time-window '{ "Mode": "OFF" }'
```

Cron-based schedules

A cron expression creates a fine-grained recurring schedule that runs at a specific time of your choosing. EventBridge Scheduler supports configuring cron-based schedules in Universal Coordinated Time (UTC), or in the time zone that you specify when you create your schedule. With cron-based schedules, you have more control over when and how often your schedule runs. Use cron-based schedules when you need a customized recurrence schedule that is not supported by one of EventBridge Scheduler's rate expressions. For example, you can create a cron-based schedule that runs at 8:00 a.m. PST on the first Monday of every month. You configure a cron-based schedule using a *cron expression*.

A cron expression consists of five required fields separated by white space: minutes, hours, day-of-month, month, day-of-week, and one optional field, year, as shown in the following.

Syntax

```
cron(minutes hours day-of-month month day-of-week year)
```

Field	Values	Wildcards
Minutes	0-59	, - * /
Hours	0-23	, - * /
Day-of-month	1-31	, - * ? / L W
Month	1-12 or JAN-DEC	, - * /
Day-of-week	1-7 or SUN-SAT	, - * ? L #
Year	1970-2199	, - * /

Wildcards

- The , (comma) wildcard includes additional values. In the Month field, JAN,FEB,MAR includes January, February, and March.
- The - (dash) wildcard specifies ranges. In the Day field, 1-15 includes days 1 through 15 of the specified month.

- The * (asterisk) wildcard includes all values in the field. In the Hours field, * includes every hour. You can't use * in both the Day-of-month and Day-of-week fields. If you use it in one, you must use ? in the other.
- The / (slash) wildcard specifies increments. In the Minutes field, you could enter 1/10 to specify every tenth minute, starting from the first minute of the hour (for example, the 11th, 21st, and 31st minute, and so on).
- The ? (question mark) wildcard specifies any. In the Day-of-month field you could enter 7 and if any day of the week was acceptable, you could enter ? in the Day-of-week field.
- The L wildcard in the Day-of-month or Day-of-week fields specifies the last day of the month or week.
- The W wildcard in the Day-of-month field specifies a weekday. In the Day-of-month field, 3W specifies the weekday closest to the third day of the month.
- The # wildcard in the Day-of-week field specifies a certain instance of the specified day of the week within a month. For example, 3#2 would be the second Tuesday of the month: the 3 refers to Tuesday because it is the third day of each week, and the 2 refers to the second day of that type within the month.

Note

If you use a '#' character, you can define only one expression in the day-of-week field. For example, "3#1, 6#3" is not valid because it is interpreted as two expressions.

Examples

The following example shows how to use cron expressions with the AWS CLI `create-schedule` command to configure a cron-based schedule. This example creates a schedule that runs at 10:15am UTC+0 on the last Friday of each month during the years 2022 to 2023, and delivers a message to an Amazon SQS queue, using the templated `SqsParameters` target type.

```
$ aws scheduler create-schedule --schedule-expression "cron(15 10 ? * 6L 2022-2023)" --  
name schedule-name \  
--target '{"RoleArn": "role-arn", "Arn": "QUEUE_ARN", "Input": "TEST_PAYLOAD" }' \  
--flexible-time-window '{ "Mode": "OFF" }'
```

One-time schedules

A one-time schedule will invoke a target only once at the date and time that you specify using a valid date, and a timestamp. EventBridge Scheduler supports scheduling in Universal Coordinated Time (UTC), or in the time zone that you specify when you create your schedule.

Note

A one-time schedule still count against your account quota *after* it has completed running and invoking its target. We recommend [deleting](#) your one-time schedules after they've completed running.

You configure a one-time schedule using an *at expression*. An at expression consists of the date and time at which you want EventBridge Scheduler to invoke your schedule, as shown in the following.

Syntax

```
at(yyyy-mm-ddThh:mm:ss)
```

When you configure a one-time schedule, EventBridge Scheduler ignores the `StartDate` and `EndDate` you specify for the schedule.

Examples

The following example shows how to use at expressions with the AWS CLI `create-schedule` command to configure a one-time schedule. This example creates a schedule that runs once at 1pm UTC-8 on November 20, 2022, and delivers a message to an Amazon SQS queue, using the templated `SqsParameters` target type.

```
$ aws scheduler create-schedule --schedule-expression "at(2022-11-20T13:00:00)" --  
name schedule-name \  
--target '{"RoleArn": "role-arn", "Arn": "QUEUE_ARN", "Input": "TEST_PAYLOAD" }' \  
--schedule-expression-timezone "America/Los_Angeles"  
--flexible-time-window '{ "Mode": "OFF" }'
```

Time zones on EventBridge Scheduler

EventBridge Scheduler supports configuring cron-based and one-time schedules in any time zone that you specify. EventBridge Scheduler uses the [Time Zone Database](#) maintained by the Internet Assigned Numbers Authority (IANA).

With the AWS CLI, you can set the time zone in which you want EventBridge Scheduler to evaluate your schedule using the `--schedule-expression-timezone` parameter. For example, the following command creates a cron-based schedule that invokes a templated Amazon SQS `SendMessage` target in **America/New_York** every day at 8:30 a.m.

```
$ aws scheduler create-schedule --schedule-expression "cron(30 8 * * ? *)" --name
schedule-in-est \
  --target '{"RoleArn": "role-arn", "Arn": "QUEUE_ARN", "Input": "This schedule runs
in the America/New_York time zone." }' \
  --schedule-expression-timezone "America/New_York"
  --flexible-time-window '{ "Mode": "OFF" }'
```

Daylight savings time on EventBridge Scheduler

EventBridge Scheduler automatically adjusts your schedule for daylight saving time. When time shifts *forward* in the Spring, if a cron expression falls on a non-existent date and time, your schedule invocation is skipped. When time shifts *backwards* in the Fall, your schedule runs only once and does not repeat its invocation. The following invocations occur normally at the specified date and time.

EventBridge Scheduler adjusts your schedule depending on the time zone you specify when you create the schedule. If you configure a schedule in **America/New_York**, your schedule adjusts when the time changes in that time zone, while a schedule in **America/Los_Angeles** is adjusted three hours later when the time changes on the west coast.

For rate-based schedules that use days as the unit, such as `rate(1 days)`, days represents a 24-hour duration on the clock. This means that when daylight savings time causes a day to shorten to 23 hours, or extend to 25 hours, EventBridge Scheduler still evaluates the rate expression 24 hours after the schedule's last invocation.

 Note

Some time zones do not observe daylight savings time, according to local rules and regulations. If you create a schedule in a time zone that does not observe daylight savings time, EventBridge Scheduler does not adjust your schedule. Daylight-savings time adjustments do not apply to schedules in universal coordinated time (UTC).

Example

Consider a scenario where you create a schedule using the following cron expression in **America/Los_Angeles**: `cron(30 2 * * ? *)`. This schedule runs every day at 2:30 a.m. in the specified time zone.

- **Spring-forward** – When time shifts forward in the Spring from 1:59 a.m. to 3:00 a.m., EventBridge Scheduler skips the schedule invocation on that day, and resumes running the schedule normally the following day.
- **Fall-back** – When time shifts backwards in the Fall from 2:59 a.m. to 2:00 a.m., EventBridge Scheduler runs the schedule only once at 2:30 a.m. before the shift occurs, but does not repeat the schedule invocation again at 2:30 a.m. after the time shift.

Managing a schedule in EventBridge Scheduler

A *schedule* is the main resource you create, configure, and manage using Amazon EventBridge Scheduler.

Every schedule has a *schedule expression* that determines when, and with what frequency, the schedule runs. EventBridge Scheduler supports three types of schedules: rate, cron, and one-time schedules. For more information about different schedule types, see [Schedule types](#).

When you create a schedule, you configure a target for the schedule to invoke. A target is an API operation that EventBridge Scheduler calls on your behalf every time your schedule runs. EventBridge Scheduler supports two types of targets: *templated* targets call common API operations across a core groups of services, and the *universal target parameter (UTP)* that you can use to call more than 6,000 operations across over 270 services. For more information about configuring targets, see [Managing targets](#).

You configure how your schedule handles failures, when EventBridge Scheduler is unable to deliver an event successfully to a target, by using two primary mechanisms: a *retry policy*, and a *dead-letter queue (DLQ)*. A retry policy determines the number of times EventBridge Scheduler must retry a failed event, and how long to keep an unprocessed event. A DLQ is a standard Amazon SQS queue EventBridge Scheduler uses to deliver failed events to, after the retry policy has been exhausted. You can use a DLQ to troubleshoot issues with your schedule or its downstream target. For more information about, see [the section called “Configuring a DLQ”](#).

In this section, you can find examples for managing your EventBridge Scheduler schedules using the console, the AWS CLI and the EventBridge Scheduler SDKs.

Topics

- [Changing the schedule state in EventBridge Scheduler](#)
- [Configuring flexible time windows in EventBridge Scheduler](#)
- [Configuring a schedule's dead-letter queue in EventBridge Scheduler](#)
- [Deleting a schedule in EventBridge Scheduler](#)
- [What's next?](#)

Changing the schedule state in EventBridge Scheduler

An EventBridge Scheduler schedule has two states: *enabled* and *disabled*. The following example uses `UpdateSchedule` to disable a schedule that fires every five minutes and invokes a Lambda target.

When you use `UpdateSchedule`, you must provide all required parameters. EventBridge Scheduler replaces your schedule with the information you provide. If you do not specify a parameter that you've previously set, it defaults to `null`.

Example AWS CLI

```
$ aws scheduler update-schedule --name lambda-universal --schedule-expression 'rate(5
minutes)' \
--target '{"RoleArn": "ROLE_ARN", "Arn": "arn:aws:scheduler::aws-sdk:lambda:invoke"
"Input": "{\"FunctionName\": \"arn:aws:lambda:REGION:123456789012:function:HelloWorld
\", \"InvocationType\": \"Event\", \"Payload\": \"{\\\"message\\\": \\\"testing function\\\"
}\\\"}\" }' \
--flexible-time-window '{ "Mode": "OFF"}' \
--state DISABLED
```

```
{
  "ScheduleArn": "arn:aws:scheduler:us-west-2:123456789012:schedule/default/lambda-
universal"
}
```

The following example uses the Python SDK and the `UpdateSchedule` operation to disable a schedule that targets Amazon SQS using a templated target.

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

sqs_templated = {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "<QUEUE_ARN>",
    "Input": "{}"

flex_window = { "Mode": "OFF" }
```



```
scheduler.update_schedule(Name="your-schedule",
    ScheduleExpression="rate(5 minutes)",
    Target=sqs_templated,
    FlexibleTimeWindow=flex_window,
    State='DISABLED')
```

Configuring flexible time windows in EventBridge Scheduler

When you configure your schedule with a flexible time window, EventBridge Scheduler invokes the target within the time window you set. This is useful in cases that do not require precise scheduled invocation of targets. Setting a flexible time window improves the reliability of your schedule by dispersing your target invocations.

For example, if you configure a 15 minute flexible time window for a schedule that runs every hour, it invokes the target within 15 minutes after the scheduled time. The following AWS CLI, and EventBridge Scheduler SDK examples use `UpdateSchedule` to set a 15 minute flexible time window for a schedule that runs once every hour.

Note

You must specify whether you want to set a flexible time window or not. If you do not want to set this option, specify `OFF`. If you do set the value to `FLEXIBLE`, you must then specify a maximum window of time during which you schedule will run.

Example AWS CLI

```
$ aws scheduler update-schedule --name lambda-universal --schedule-expression 'rate(1
hour)' \
--target '{"RoleArn": "ROLE_ARN", "Arn": "arn:aws:scheduler::aws-sdk:lambda:invoke"
"Input": "{\\"FunctionName\\":\\"arn:aws:lambda:REGION:123456789012:function:HelloWorld
\\",\\"InvocationType\\":\\"Event\\",\\"Payload\\":\\"{\\\\"message\\\\":\\\\"testing function\\\\"
\\"}\\\\"}" }' \
--flexible-time-window '{ "Mode": "FLEXIBLE", "MaximumWindowInMinutes": 15} \
```

```
{
    "ScheduleArn": "arn:aws:scheduler:us-west-2:123456789012:schedule/lambda-universal"
}
```

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

sqs_templated = {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "<QUEUE_ARN>",
    "Input": "{}"}

flex_window = { "Mode": "FLEXIBLE", "MaximumWindowInMinutes": 15}

scheduler.update_schedule(Name="your-schedule",
    ScheduleExpression="rate(1 hour)",
    Target=sqs_templated,
    FlexibleTimeWindow=flex_window)
```

Configuring a schedule's dead-letter queue in EventBridge Scheduler

Amazon EventBridge Scheduler supports dead-letter queues (DLQ) using Amazon Simple Queue Service. When a schedule fails to invoke its target, EventBridge Scheduler delivers a JSON payload containing invocation details and any response received from the target to an Amazon SQS standard queue that you specify.

The following topic refers to this JSON as a *dead-letter event*. A dead-letter event lets you troubleshoot issues with your schedule or targets. If you configure a retry policy for your schedule, EventBridge Scheduler delivers the dead-letter event it has exhausting the maximum number of retries you set.

The following topics describe how you can configure an Amazon SQS queue as a DLQ for your schedule, set up the permissions EventBridge Scheduler needs to deliver messages to Amazon SQS, and receive dead-letter events from the DLQ.

Topics

- [Create an Amazon SQS queue](#)
- [Set up execution role permissions](#)
- [Specify a dead-letter queue](#)

- [Retrieve the dead-letter event](#)

Create an Amazon SQS queue

Before you configure a DLQ for your schedule, you must create a standard Amazon SQS queue. For instructions on creating a queue using the Amazon SQS console, see [Creating an Amazon SQS queue](#) in the *Amazon Simple Queue Service Developer Guide*.

Note

EventBridge Scheduler does not support using a FIFO queue as your schedule's DLQ.

Use the following AWS CLI command to create a standard queue.

```
$ aws sqs create-queue --queue-name queue-name
```

If successful, you'll see the QueueURL in the output.

```
{
  "QueueUrl": "https://sqs.us-west-2.amazonaws.com/123456789012/scheduler-dlq-test"
}
```

After you've created the queue, note the queue ARN. You'll need the ARN when you specify a DLQ for your EventBridge Scheduler schedule. You can find your queue ARN in the Amazon SQS console, or by using the [get-queue-attributes](#) AWS CLI command.

```
$ aws sqs get-queue-attributes --queue-url your-dlq-url --attribute-names QueueArn
```

If successful, you will see the queue ARN in the output.

```
{
  "Attributes": {
    "QueueArn": "arn:aws:sqs:us-west-2:123456789012:scheduler-dlq-test"
  }
}
```

In the next section, you will add the required permissions to your schedule execution role to allow EventBridge Scheduler to deliver dead-letter events to Amazon SQS.

Set up execution role permissions

To let EventBridge Scheduler to deliver dead-letter events to Amazon SQS, your schedule execution role needs the following permission policy. For more information on attaching a new permission policy to your schedule execution role, see [Setting up the execution role](#).

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sqs:SendMessage"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Note

Your schedule execution role might already have the required permissions attached if you use EventBridge Scheduler to invoke an Amazon SQS API target.

In the next section, you'll use the EventBridge Scheduler console and specify a DLQ for your schedule.

Specify a dead-letter queue

To specify a DLQ, use the EventBridge Scheduler console or the AWS CLI to update an existing schedule, or create a new one.

Console

To specify a DLQ using the console

1. Sign in to the AWS Management Console, then choose the following link to open the EventBridge Scheduler section of the EventBridge console: <https://console.aws.amazon.com/scheduler/home>

2. On the EventBridge Scheduler console, create a new schedule, or choose an existing schedule from your list of schedules to edit.
3. On the **Settings** page, for **Dead-letter queue (DLQ)**, do one of the following:
 - Choose **Select an Amazon SQS queue in my AWS account as a DLQ**, then choose the queue ARN for your DLQ from the dropdown list.
 - Choose **Specify an Amazon SQS queue in other AWS accounts as a DLQ**, then enter the queue ARN for your DLQ. If you choose a queue in another AWS account, the EventBridge Scheduler console will not be able to display the queue ARNs in a dropdown list.
4. Review your selections, then choose **Create schedule** or **Save schedule** to finish configuring a DLQ.
5. (Optional) To view a schedule's DLQ details, choose the name of the schedule from the list, then choose the **Dead-letter queue** tab on the **Schedule detail** page.

AWS CLI

To update an existing schedule using the AWS CLI

- Use the [update-schedule](#) command to update your schedule. Specify the Amazon SQS queue you created previously as the DLQ. Specify the IAM role ARN to which you attached the required Amazon SQS permissions as the execution role. Replace all other placeholder values with your information.

```
$ aws scheduler update-schedule --name existing-schedule \
  --schedule-expression 'rate(5 minutes)' \
  --target '{"DeadLetterConfig": {"Arn": "DLQ_ARN"}, "RoleArn": "ROLE_ARN",
  "Arn": "QUEUE_ARN", "Input": "Hello world!" }' \
  --flexible-time-window '{ "Mode": "OFF" }'
```

To create a new schedule with a DLQ using the AWS CLI

- Use the [create-schedule](#) command to create a schedule. Replace all placeholder values with your information.

```
$ aws scheduler create-schedule --name new-schedule \
  --schedule-expression 'rate(5 minutes)' \
```

```
--target '{"DeadLetterConfig": {"Arn": "DLQ_ARN"}, "RoleArn": "ROLE_ARN",
"Arn": "QUEUE_ARN", "Input": "Hello world!" }' \
--flexible-time-window '{ "Mode": "OFF"}
```

In the next section, you'll use the AWS CLI to retrieve a dead-letter event from the DLQ.

Retrieve the dead-letter event

Use the [receive-message](#) command, as shown in the following, to retrieve a dead-letter event from the DLQ. You can set the number of messages to retrieve using the `--max-number-of-messages` attribute.

```
$ aws sqs receive-message --queue-url your-dlq-url --attribute-names All --message-
attribute-names All --max-number-of-messages 1
```

If successful, you will see output similar to the following.

```
{
  "Messages": [
    {
      "MessageId": "2aeg3510-fe3a-4f5a-ab6a-6906560eaf7e",
      "ReceiptHandle": "AQEBkNKTdOMrWgHKPoITRBwrPoK3eCSZICZwVqCY0BZ
+FfTcORFpopJbtCqj36VbBT1HreM8+qM/m5jcwqSlAlGmIJ0/hYmMgn/
+dwIty9izE7HnpvRhhEyHxbeTZ5V05RbeasYaBdNyi9WLcnAHviDh6MebLXXNWoFyYNSxdwJuG0f/
w3htX6r3dpxXvvFNPGoQb8ihY37+u0gtsbuIwhLtUSmE8rbldEEwiUfi3IJ1zEZpUS77n/k1GWrMrnYg0Gx/
BuaLz0rFi2F738XI/
Hnh45uv3ca60YwS1ojPQ1LtX2URg1haV5884FY1aRvY8jRlpCZabTkYRTZKSXG5KNGYZnHpmsspii6JNkjitYVFKPo0H91w
      "MD5OfBody": "07adc3fc889d6107d8bb8fda42fe0573",
      "Body": "{\"MessageBody\": \"Hello, world!\", \"QueueUrl\": \"https://sqs.us-
west-2.amazonaws.com/123456789012/does-not-exist\"}\",
      "Attributes": {
        "SenderId": "ARO0A2DZE3W4CTL5ZR7EIN:ff00212d8c453aaaae644bc6846d4723",
        "ApproximateFirstReceiveTimestamp": "1652499058144",
        "ApproximateReceiveCount": "2",
        "SentTimestamp": "1652490733042"
      },
      "MD5OfMessageAttributes": "f72c1d78100860e00403d849831d4895",
      "MessageAttributes": {
        "ERROR_CODE": {
          "StringValue": "AWS.SimpleQueueService.NonExistentQueue",
          "DataType": "String"
```

```

    },
    "ERROR_MESSAGE": {
      "StringValue": "The specified queue does not exist for this wsdl
version.",
      "DataType": "String"
    },
    "EXECUTION_ID": {
      "StringValue": "ad06616e51cdf74a",
      "DataType": "String"
    },
    "EXHAUSTED_RETRY_CONDITION": {
      "StringValue": "MaximumEventAgeInSeconds",
      "DataType": "String"
    }
  }
  "IS_PAYLOAD_TRUNCATED": {
    "StringValue": "false",
    "DataType": "String"
  },
  "RETRY_ATTEMPTS": {
    "StringValue": "0",
    "DataType": "String"
  },
  "SCHEDULED_TIME": {
    "StringValue": "2022-05-14T01:12:00Z",
    "DataType": "String"
  },
  "SCHEDULE_ARN": {
    "StringValue": "arn:aws:scheduler:us-west-2:123456789012:schedule/
DLQ-test",
    "DataType": "String"
  },
  "TARGET_ARN": {
    "StringValue": "arn:aws:scheduler::aws-sdk:sqs:sendMessage",
    "DataType": "String"
  }
}
]
}

```

Note the following attributes in the dead-letter event to help you identify and troubleshoot possible reasons why target invocation has failed.

- **ERROR_CODE** – Contains the error code that EventBridge Scheduler receives from the target's service API. In the preceding example, the error code returned by Amazon SQS is `AWS.SimpleQueueService.NonExistentQueue`. If the schedule fails to invoke a target due to an issue with EventBridge Scheduler, you'll see the following error code instead: `AWS.Scheduler.InternalServerError`.
- **ERROR_MESSAGE** – Contains the error message that EventBridge Scheduler receives from the target's service API. In the preceding example, the error message returned by Amazon SQS is `The specified queue does not exist for this wsdl version`. If the schedule fails due to an issue with EventBridge Scheduler, you'll see the following error message instead: `Unexpected error occurred while processing the request`.
- **TARGET_ARN** – The ARN of the target that your schedule invokes, in the following service ARN format: `arn:aws:scheduler::aws-sdk:service:apiAction`.
- **EXHAUSTED_RETRY_CONDITION** – Indicates why the event was delivered to the DLQ. This attribute will be present if the error from the target API is a retryable error, and not a permanent error. The attribute can contain the values `MaximumRetryAttempts` if EventBridge Scheduler sent it to the DLQ after it exceeded the maximum retry attempts you configured for the schedule, or `MaximumEventAgeInSeconds`, if the event is older than the maximum age you configured on the schedule and is still failing to deliver.

In the preceding example, we can determine, based on the error code, and the error message, that the target queue we specified for the schedule does not exist.

Deleting a schedule in EventBridge Scheduler

You can delete a schedule by either configuring automatic deletion, or by manually deleting an individual schedule. Use following topics to learn how you to delete a schedule using both methods, and why you might choose one method over the other.

Topics

- [Deletion after schedule completion](#)
- [Manual deletion](#)

Deletion after schedule completion

Configure automatic deletion after schedule completion if you want to avoid having to individually manage your schedule resources on EventBridge Scheduler. In applications where you create thousands of schedules at a time and need flexibility to scale up the number of your schedules on demand, automatic deletion can ensure that you do not reach your account quota for the [number of schedules](#) in a specified Region.

When you configure automatic deletion for a schedule, EventBridge Scheduler deletes the schedule after its last target invocation. For one-time schedules, this occurs after the schedule has invoked its target once. For recurring schedules you set up with rate, or cron, expressions, your schedule is deleted after its last invocation. A recurring schedule's last invocation is the invocation that occurs closest to the [EndDate](#) you specify. If you configure a schedule with automatic deletion but do not specify a value for EndDate, EventBridge Scheduler does not automatically delete the schedule.

You can set up automatic deletion when you first create a schedule, or update preferences for an existing schedule. The following steps describe how to configure automatic deletion for an existing schedule.

AWS Management Console

1. Open the EventBridge Scheduler console at <https://console.aws.amazon.com/scheduler/>.
2. From the list of schedules, select the schedule you want to edit, then choose **Edit**.
3. From the navigation list on the left, choose **Settings**.
4. In the **Action after schedule completion** section, select **DELETE** from the drop down list, then save your changes.

AWS CLI

1. Open a new prompt window.
2. Use the [update-schedule](#) AWS CLI command to update an existing schedule as shown in the following. The command sets the `--action-after-completion` to `DELETE`. This example assumes that you have defined your target configuration locally in a JSON file. To update a schedule, you must provide the target, as well as any other schedule parameters you want to configure for your existing schedule.

This is a recurring schedule with a rate of one invocation per hour. Therefore, you specify an end date when setting the `--action-after-completion` parameter.

```
$ aws scheduler update-schedule --name schedule-name \
--action-after-completion 'DELETE' \
--schedule-expression 'rate(1 hour)' \
--end-date '2024-01-01T00:00:00' \
--target file://target-configuration.json \
--flexible-time-window '{ "Mode": "OFF" }' \
```

Manual deletion

When you no longer need a schedule, you can delete it using the [DeleteSchedule](#) operation.

Example AWS CLI

```
$ aws scheduler delete-schedule --name your-schedule
```

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

scheduler.delete_schedule(Name="your-schedule")
```

What's next?

- For more information on how you can configure templated targets for Lambda and Step Functions, and to learn about using the universal target parameter, see [Managing targets](#).
- For more information about the EventBridge Scheduler data types and API operations, see the [EventBridge Scheduler API Reference](#).

Managing a schedule group in EventBridge Scheduler

A *schedule group* is an Amazon EventBridge Scheduler resource that you use to organize your schedules.

Your AWS account comes with a default scheduler group. You can associate a new schedule with the default group or with schedule groups that you create and manage. You can create up to [500 schedule groups](#) in your AWS account. With EventBridge Scheduler, you organize schedule groups, instead of individual schedules, by applying [tags](#).

A *tag* is a label comprised of a case-sensitive key and a case-sensitive value that you define. You can create tags to categorize schedules by criteria like purpose, owner, or environment. For example, you can identify the environment that your schedules belong to with the following tag: `environment:production`.

Important

Do not add personally identifiable information (PII) or other confidential or sensitive information in tags. Tags are accessible to many AWS services, including billing. Tags are not intended to be used for private or sensitive data.

A schedule group has two possible [states](#): **ACTIVE** and **DELETING**.

When you first create a group, it is **ACTIVE** by default. You can add schedules to an **ACTIVE** group. When you delete a group, the state changes to **DELETING** until EventBridge Scheduler finishes the deletion of the associated schedules. After EventBridge Scheduler deletes the schedules in the group, the group is no longer available in your account.

Use the following topics to create a schedule group and apply a tag to it. You'll also associate a schedule with the group. Finally, you'll delete the group.

Topics

- [Creating a schedule group in EventBridge Scheduler](#)
- [Deleting a schedule group in EventBridge Scheduler](#)
- [Related resources](#)

Creating a schedule group in EventBridge Scheduler

Use schedule groups and tagging to organize schedules that share a common purpose or belong to the same environment. In the following steps, you create a new schedule group and label it using a tag. You then associate a new schedule with that group.

Note

Once you create a group, you can't remove a schedule from that group, or associate the schedule with a different group. You can only associate a schedule with a group when you first create the schedule.

Step one: Create a new schedule group

The following topics describe how to create a new schedule group and label it with the following tag: `environment:development`.

AWS Management Console

To create a new group using the AWS Management Console

1. Sign in to the AWS Management Console and open the Amazon EventBridge console at <https://console.aws.amazon.com/events/>.
2. In the left navigation pane, choose **Schedule groups**.
3. On the Schedule groups page, choose **Create schedule group**.
4. In the **Schedule group detail** section, for **Name**, enter a name for the group. For example, **TestGroup**.
5. In the **Tags** section, do the following:
 - a. Choose **Add new tag**.
 - b. For **Key**, enter the name that you want to assign to this key. For this tutorial, to label the environment this schedule group belongs to, enter **environment**.
 - c. For **Value - optional**, enter the value that you want to assign to this key. For this tutorial, enter the value **development** for your environment key.

Note

You can add additional tags to your group after you create it.

6. To finish, choose **Create schedule group**. Your new group appears in the **Schedule groups** list.
7. (Optional) To edit a group or manage its tags, select the check box for the new group and choose **Edit**.

Note

You can't edit the default schedule group.

AWS CLI

To create a new group using the AWS CLI

1. Open a new command prompt window.
2. From the AWS Command Line Interface (AWS CLI), enter the following [create-schedule-group](#) command to create a new group. This command creates a group with one tag: `environment:development`. You can use this tag or a similar tagging system to label your schedule groups according to the environment they belong to.

Replace the schedule name and the tag key and value with your information.

```
$ aws scheduler create-schedule-group --name TestGroup --tags  
Key=environment,Value=development
```

By default, your new group is in the ACTIVE state. You can now associate new schedules with the new group you created.

Step two: Associating a schedule with the group

Use the following steps to associate a new schedule with the group you created in the [previous step](#).

AWS Management Console

To associate a schedule with a group using the AWS Management Console

1. Sign in to the AWS Management Console and open the Amazon EventBridge console at <https://console.aws.amazon.com/events/>.
2. In the left navigation pane, choose **Schedules** in the left navigation pane.
3. From the **Schedules** table, choose **Create schedule** to create a new schedule.
4. On the **Specify schedule details** page, for **Schedule group**, select the name of your new group from the drop down list. For example, select TestGroup.
5. Specify a schedule pattern, target, settings then review your selection on the **Review and save schedule** page. For more information on configuring a new schedule, see [Getting started](#).
6. To finish and save your schedule, choose **Save schedule**.

AWS CLI

To associate a schedule with a group using the AWS CLI

1. Open a new command prompt window.
2. From the AWS Command Line Interface (AWS CLI), enter the following [create-schedule](#) command. This creates a schedule and associates it with the group from the [previous step](#), named sqs-test-schedule. This schedule uses the templated [Amazon SQS](#) target type to invoke the SendMessage operation. Replace the schedule name, target, and group name with your information.

```
$ aws scheduler create-schedule --name sqs-test-schedule --schedule-expression  
'rate(5 minutes)' \  
--target '{"RoleArn": "ROLE_ARN", "Arn": "QUEUE_ARN", "Input": "TEST_PAYLOAD" }'  
\  
--group-name TestGroup  
--flexible-time-window '{ "Mode": "OFF" }'
```

Your new schedule is now associated with the TestGroup schedule group.

Deleting a schedule group in EventBridge Scheduler

In the following, you can learn how to delete a schedule group using the AWS Management Console and the AWS Command Line Interface. When you delete a group, it is in the **DELETING** state until EventBridge Scheduler deletes all schedules in the group. After EventBridge Scheduler deletes the schedules in the group, the group is no longer available in your account.

Note

Once you create a group, you can't remove a schedule from that group, or associate the schedule with a different group. You can only associate a schedule with a group when you first create the schedule.

AWS Management Console

To delete a group using the AWS Management Console

1. Sign in to the AWS Management Console and open the Amazon EventBridge console at <https://console.aws.amazon.com/events/>.
2. In the left navigation pane, choose **Schedule groups** in the left navigation pane.
3. On the **Schedule groups** page, from the list of existing groups in the current AWS Region, locate the group you want to delete. If you don't see the group you're looking for, choose another AWS Region.

Note

You can't delete, or edit, the **default** group.

4. Select the check box for the group that you want to delete.
5. Choose **Delete**.
6. In the **Delete schedule group** dialog box, enter the name of the group to confirm your choice, then choose **Delete**.
7. In the **Schedule groups** list, the **Status** column changes to indicate that your group is now **Deleting**. The group remains in this state until EventBridge Scheduler deletes all of the schedules associated with the group.
8. To refresh the list and confirm that the group was deleted, choose the **Refresh** icon.

AWS CLI

To delete a group using the AWS CLI

1. Open a new command prompt window.
2. From the AWS Command Line Interface (AWS CLI), enter the following [delete-schedule-group](#) command to delete the schedule group. Replace the value for `--name` with your information.

```
$ aws scheduler delete-schedule-group --name TestGroup
```

If successful, this AWS CLI operation doesn't return a response.

3. To verify that the group is in the DELETING state, run the following [get-schedule-group](#) command.

```
$ aws scheduler get-schedule-group --name TestGroup
```

If successful, you receive output similar to the following:

```
{
  "Arn": "arn:aws::scheduler:us-west-2:123456789012:schedule-group/TestGroup",
  "CreationDate": "2023-01-01T09:00:00.000000-07:00",
  "LastModificationDate": "2023-01-01T09:00:00.000000-07:00",
  "Name": "TestGroup",
  "State": "DELETING"
}
```

EventBridge Scheduler deletes the group after it deletes the schedules associated with the group. If you run `get-schedule-group` again, you receive following `ResourceNotFoundException` response:

```
An error occurred (ResourceNotFoundException) when calling the GetScheduleGroup
operation: Schedule group TestGroup does not exist.
```

Related resources

For more information on schedule groups, see the following resources:

- [CreateScheduleGroup](#) operation in the *EventBridge Scheduler API Reference*.
- [DeleteScheduleGroup](#) operation in the *EventBridge Scheduler API Reference*.

Managing targets in EventBridge Scheduler

The following topics describe how to use templated, and universal targets with EventBridge Scheduler, and provides a list of supported AWS services that you can configure using EventBridge Scheduler's universal target parameter.

Templated targets are a set of common API operations across a group of core AWS services such as Amazon SQS, Lambda, and Step Functions. For example, you can target Lambda's [Invoke](#) API operation by providing the function ARN, or Amazon SQS's [SendMessage](#) operation with the queue ARN of the target.

The *universal target* is a customizable set of parameters that allow you to invoke a wider set of API operation for many AWS service. For example, you can use EventBridge Scheduler's universal target parameter (UTP) to create a new Amazon SQS queue using the [CreateQueue](#) operation.

To configure either templated or universal targets, your schedule must have permission to call the API operation that you configure as your target. You do this by attaching the required permissions to your schedule's execution role. For example, to target Amazon SQS's [SendMessage](#) operation, the execution role be granted permission to perform the `sqs:SendMessage` action. In most cases, you can add the necessary permissions by using the [AWS managed policies](#) that the target service supports. However, you can also create your own [customer managed policies](#), or add [inline permissions](#) to an existing policy attached to the execution role. The following topics demonstrate examples of adding permissions for both templated, and universal, target types.

For more information about setting up an execution role for a schedule, see [the section called "Set up the execution role"](#).

Topics

- [Using templated targets in EventBridge Scheduler](#)
- [Using universal targets in EventBridge Scheduler](#)
- [Adding context attributes in EventBridge Scheduler](#)
- [What's next?](#)

Using templated targets in EventBridge Scheduler

Templated targets are a set of common API operations across a group of core AWS services, such as Amazon SQS, Lambda, and Step Functions. For example, you can target Lambda's [Invoke](#)

operation by providing the function ARN, or Amazon SQS's [SendMessage](#) operation using the queue ARN. To configure a templated target, you must also grant permissions to the schedule's execution role to perform the targeted API operation.

To configure a templated target programatically using the AWS CLI or one of the EventBridge Scheduler SDKs, you need to specify the ARN of the execution role, the ARN for target resource, an optional input that you want EventBridge Scheduler to deliver to the target, and for some templated targets, a unique set of parameters with additional configuration options for that target. When you specify the ARN for a templated target resource, EventBridge Scheduler automatically assumes that you want to call the supported API operation for that service. If you want EventBridge Scheduler to target a different API operation for the service, you must configure the target as a [universal target](#).

The following is a complete list of all templated targets that EventBridge Scheduler supports, and if applicable, each target's unique set of associated parameters. Choose the link for each parameter set to see the required, and optional, fields in the *EventBridge Scheduler API Reference*.

- **CodeBuild** – [StartBuild](#)
- **CodePipeline** – [StartPipelineExecution](#)
- **Amazon ECS** – [RunTask](#)
 - Parameters: [EcsParameters](#)
- **EventBridge** – [PutEvents](#)
 - Parameters: [EventBridgeParameters](#)
- **Amazon Inspector** – [StartAssessmentRun](#)
- **Kinesis** – [PutRecord](#)
 - Parameters: [KinesisParameters](#)
- **Firehose** – [PutRecord](#)
- **Lambda** – [Invoke](#)
- **SageMaker AI** – [StartPipelineExecution](#)
 - Parameters: [SageMakerPipelineParameters](#)
- **Amazon SNS** – [Publish](#)
- **Amazon SQS** – [SendMessage](#)
 - Parameters: [SqsParameters](#)
- **Step Functions** – [StartExecution](#)

Use the following examples to learn how to configure different templated targets, and the required IAM permissions for each described target.

Amazon SQS SendMessage

Example Permission policy for execution role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sqs:SendMessage"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Example AWS CLI

```
$ aws scheduler create-schedule --name sqs-templated --schedule-expression 'rate(5
minutes)' \
--target '{"RoleArn": "ROLE_ARN", "Arn": "QUEUE_ARN", "Input": "Message for scheduleArn:
'<aws.scheduler.schedule-arn>', scheduledTime: '<aws.scheduler.scheduled-time>'"}' \
--flexible-time-window '{"Mode": "OFF"}'
```

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

flex_window = { "Mode": "OFF" }

sqs_templated = {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "<QUEUE_ARN>",
    "Input": "Message for scheduleArn: '<aws.scheduler.schedule-arn>', scheduledTime:
'<aws.scheduler.scheduled-time>'"}
}
```

```
scheduler.create_schedule(  
    Name="sqs-python-templated",  
    ScheduleExpression="rate(5 minutes)",  
    Target=sqs_templated,  
    FlexibleTimeWindow=flex_window)
```

Example Java SDK

```
package com.example;  
  
import software.amazon.awssdk.regions.Region;  
import software.amazon.awssdk.services.scheduler.SchedulerClient;  
import software.amazon.awssdk.services.scheduler.model.*;  
  
public class MySchedulerApp {  
  
    public static void main(String[] args) {  
  
        final SchedulerClient client = SchedulerClient.builder()  
            .region(Region.US_WEST_2)  
            .build();  
  
        Target sqsTarget = Target.builder()  
            .roleArn("<ROLE_ARN>")  
            .arn("<QUEUE_ARN>")  
            .input("Message for scheduleArn: '<aws.scheduler.schedule-arn>'",  
scheduledTime: '<aws.scheduler.scheduled-time>')  
            .build();  
  
        CreateScheduleRequest createScheduleRequest = CreateScheduleRequest.builder()  
            .name("<SCHEDULE_NAME>")  
            .scheduleExpression("rate(10 minutes)")  
            .target(sqsTarget)  
            .flexibleTimeWindow(FlexibleTimeWindow.builder()  
                .mode(FlexibleTimeWindowMode.OFF)  
                .build())  
            .build();  
  
        client.createSchedule(createScheduleRequest);  
        System.out.println("Created schedule with rate expression and an Amazon SQS  
templated target");  
    }  
}
```

```
}
```

Lambda Invoke

Example Permission policy for execution role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "lambda:InvokeFunction"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Example AWS CLI

```
$ aws scheduler create-schedule --name lambda-templated-schedule --schedule-expression
'rate(5 minutes)' \
--target '{"RoleArn": "<ROLE_ARN>", "Arn": "<LAMBDA_ARN>", "Input": "{ \"Payload\":
\"TEST_PAYLOAD\" }" }' \
--flexible-time-window '{ "Mode": "OFF" }'
```

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

flex_window = { "Mode": "OFF" }

lambda_templated = {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "<LAMBDA_ARN>",
    "Input": "{ 'Payload': 'TEST_PAYLOAD' }"
}

scheduler.create_schedule(
```

```
Name="lambda-python-templated",
ScheduleExpression="rate(5 minutes)",
Target=lambda_templated,
FlexibleTimeWindow=flex_window)
```

Example Java SDK

```
package com.example;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.scheduler.SchedulerClient;
import software.amazon.awssdk.services.scheduler.model.*;

public class MySchedulerApp {

    public static void main(String[] args) {

        final SchedulerClient client = SchedulerClient.builder()
            .region(Region.US_WEST_2)
            .build();

        Target lambdaTarget = Target.builder()
            .roleArn("<ROLE_ARN>")
            .arn("<Lambda ARN>")
            .input("{ 'Payload': 'TEST_PAYLOAD' }")
            .build();

        CreateScheduleRequest createScheduleRequest = CreateScheduleRequest.builder()
            .name("<SCHEDULE_NAME>")
            .scheduleExpression("rate(10 minutes)")
            .target(lambdaTarget)
            .flexibleTimeWindow(FlexibleTimeWindow.builder()
                .mode(FlexibleTimeWindowMode.OFF)
                .build())
            .clientToken("<Token GUID>")
            .build();

        client.createSchedule(createScheduleRequest);
        System.out.println("Created schedule with rate expression and Lambda templated
target");
    }
}
```

Step Functions StartExecution

Example Permission policy for execution role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "states:StartExecution"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

Example AWS CLI

```
$ aws scheduler create-schedule --name sfn-templated-schedule --schedule-expression
'rate(5 minutes)' \
--target '{"RoleArn": "ROLE_ARN", "Arn": "STATE_MACHINE_ARN", "Input": "{ \"Payload\":
\"TEST_PAYLOAD\" }" }' \
--flexible-time-window '{ "Mode": "OFF" }'
```

Example Python SDK

```
import boto3
scheduler = boto3.client('scheduler')

flex_window = { "Mode": "OFF" }

sfn_templated= {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "<STATE_MACHINE_ARN>",
    "Input": "{ 'Payload': 'TEST_PAYLOAD' }"
}

scheduler.create_schedule(Name="sfn-python-templated",
    ScheduleExpression="rate(5 minutes)",
    Target=sfn_templated,
    FlexibleTimeWindow=flex_window)
```


Example Java SDK

```
package com.example;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.scheduler.SchedulerClient;
import software.amazon.awssdk.services.scheduler.model.*;

public class MySchedulerApp {

    public static void main(String[] args) {

        final SchedulerClient client = SchedulerClient.builder()
            .region(Region.US_WEST_2)
            .build();

        Target stepFunctionsTarget = Target.builder()
            .roleArn("<ROLE_ARN>")
            .arn("<STATE_MACHINE_ARN>")
            .input("{ 'Payload': 'TEST_PAYLOAD' }")
            .build();

        CreateScheduleRequest createScheduleRequest = CreateScheduleRequest.builder()
            .name("<SCHEDULE_NAME>")
            .scheduleExpression("rate(10 minutes)")
            .target(stepFunctionsTarget)
            .flexibleTimeWindow(FlexibleTimeWindow.builder()
                .mode(FlexibleTimeWindowMode.OFF)
                .build())
            .clientToken("<Token GUID>")
            .build();

        client.createSchedule(createScheduleRequest);
        System.out.println("Created schedule with rate expression and Step Function
templated target");
    }
}
```

Using universal targets in EventBridge Scheduler

A *universal target* is a customizable set of parameters that allow you to invoke a wider set of API operation for many AWS services. For example, you can use a universal target parameter (UTP) to create a new Amazon SQS queue using the [CreateQueue](#) operation.

To configure a universal target for your schedule using the AWS CLI, or one of the EventBridge Scheduler SDKs, you need to specify the following information:

- **RoleArn** – The ARN for the execution role you want to use for the target. The execution role you specify must have the permissions to call the API operation you want your schedule to target.
- **Arn** – The complete service ARN, including the API operation you want to target, in the following format: `arn:aws:scheduler::aws-sdk:service:apiAction`.

For example, for Amazon SQS, the service name you specify is `arn:aws:scheduler::aws-sdk:sqs:sendMessage`.

- **Input** – A well-formed JSON you specify with the request parameters that EventBridge Scheduler sends to the target API. The parameters and shape of the JSON you set in Input are determined by the service API your schedule invokes. To find this information, see the API reference for the service you want to target.

Unsupported actions

EventBridge Scheduler does not support read-only API actions, such as common GET operations, that begin with the following list of prefixes:

```
get
describe
list
poll
receive
search
scan
query
select
read
lookup
discover
validate
```

```
batchGet
batchDescribe
batchRead
transactGet
adminGet
adminList
testMigration
retrieve
testConnection
translateDocument
isAuthorized
invokeModel
```

For example, the service ARN for the [GetQueueUrl](#) API action would be the following: `arn:aws:scheduler::aws-sdk:sqs:getQueueURL`. Since the API action starts with the `get` prefix, EventBridge Scheduler does not support this target. Similarly, the Amazon MQ action [ListBrokers](#) is not supported as a target because the operation begins with the prefix `list`.

Examples using the universal target

The parameters you pass in the schedule Input field depend on the request parameters that the service API you want to invoke accepts. For example, to target Lambda [Invoke](#), you can set the parameters listed in [AWS Lambda API Reference](#). This includes the optional JSON [payload](#) that you can pass to a Lambda function.

To determine the parameters you can set for different APIs, see the API reference for that service. Similar to Lambda Invoke, some APIs accept URI parameters, as well as a request body payload. In such cases, you specify the URI path parameters as well as the JSON payload in your schedule Input.

The following examples show how you to use the universal target to invoke common API operations with Lambda, Amazon SQS, and Step Functions.

Example Lambda

```
$ aws scheduler create-schedule --name lambda-universal-schedule --schedule-expression
'rate(5 minutes)' \
--target '{"RoleArn": "ROLE_ARN", "Arn": "arn:aws:scheduler::aws-sdk:lambda:invoke"
"Input": "{\\"FunctionName\\":\\"arn:aws:lambda:REGION:123456789012:function:HelloWorld
\\",\\"InvocationType\\":\\"Event\\",\\"Payload\\":\\"{\\\\"message\\\\":\\\\"testing function\\\\"
\\"}\\"}" }' \
```

```
--flexible-time-window '{ "Mode": "OFF" }'
```

Example Amazon SQS

```
import boto3
scheduler = boto3.client('scheduler')

flex_window = { "Mode": "OFF" }

sqs_universal= {
    "RoleArn": "<ROLE_ARN>",
    "Arn": "arn:aws:scheduler::aws-sdk:sqs:sendMessage",
    "Input": "{ \"MessageBody\": \"My message\", \"QueueUrl\": \"<QUEUE_URL>\" }"
}

scheduler.create_schedule(
    Name="sqs-sdk-test",
    ScheduleExpression="rate(5 minutes)",
    Target=sqs_universal,
    FlexibleTimeWindow=flex_window)
```

Example Step Functions

```
package com.example;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.scheduler.SchedulerClient;
import software.amazon.awssdk.services.scheduler.model.*;

public class MySchedulerApp {

    public static void main(String[] args) {

        final SchedulerClient client = SchedulerClient.builder()
            .region(Region.US_WEST_2)
            .build();

        Target stepFunctionsUniversalTarget = Target.builder()
            .roleArn("<ROLE_ARN>")
            .arn("arn:aws:scheduler::aws-sdk:sfn:startExecution")
            .input("{ \"Input\": \"{}\", \"StateMachineArn\": \"<STATE_MACHINE_ARN>\" }")
    }
```

```

        .build();

        CreateScheduleRequest createScheduleRequest = CreateScheduleRequest.builder()
            .name("<SCHEDULE_NAME>")
            .scheduleExpression("rate(10 minutes)")
            .target(stepFunctionsUniversalTarget)
            .flexibleTimeWindow(FlexibleTimeWindow.builder()
                .mode(FlexibleTimeWindowMode.OFF)
                .build())
            .clientToken("<Token GUID>")
            .build();

        client.createSchedule(createScheduleRequest);
        System.out.println("Created schedule with rate expression and Step Function
universal target");
    }
}

```

Adding context attributes in EventBridge Scheduler

Use of the following keywords in the payload you pass to the target to collect metadata about the schedule. EventBridge Scheduler replaces each keyword with its respective value when your schedule invokes the target.

- **<aws.scheduler.schedule-arn>** – The ARN of the schedule.
- **<aws.scheduler.scheduled-time>** – The time you specified for the schedule to invoke its target, for example, 2022-03-22T18:59:43Z.
- **<aws.scheduler.execution-id>** – The unique ID that EventBridge Scheduler assigns for each attempted invocation of a target, for example, d32c5kddcf5bb8c3.
- **<aws.scheduler.attempt-number>** – A counter that identifies the attempt number for the current invocation, for example, 1.

This example shows creating a schedule that fires every five minutes, and invokes the Amazon SQS SendMessage operation as a universal target. The message body includes the value for schedule-time.

Example AWS CLI

```
$ aws scheduler create-schedule --name your-schedule \
```

```
--schedule-expression 'rate(5 minutes)' \  
--target '{"RoleArn": "ROLE_ARN", \  
        "Arn": "arn:aws:scheduler::aws-sdk:sqs:sendMessage", \  
        "Input": "{\"MessageBody\": \"<aws.scheduler.scheduled-time>\", \"QueueUrl\": \  
\"https://sqs.us-west-2.amazonaws.com/123456789012/scheduler-cli-test\"}"}' \  
--flexible-time-window '{ "Mode": "OFF" }'
```

Example Python SDK

```
import boto3  
scheduler = boto3.client('scheduler')  
  
sqs_universal= {  
    "RoleArn": "<ROLE_ARN>",  
    "Arn": "arn:aws:scheduler::aws-sdk:sqs:sendMessage",  
    "Input": "{\"MessageBody\": \"<aws.scheduler.scheduled-time>\", \"QueueUrl\": \  
\"https://sqs.us-west-2.amazonaws.com/123456789012/scheduler-cli-test\"}"  
}  
  
flex_window = { "Mode": "OFF" }  
  
scheduler.update_schedule(Name="your-schedule",  
    ScheduleExpression="rate(5 minutes)",  
    Target=sqs_universal,  
    FlexibleTimeWindow=flex_window)
```

What's next?

For more information on the EventBridge Scheduler data types and API operations, see [EventBridge Scheduler API Reference](#).

Access Amazon EventBridge Scheduler using an interface endpoint (AWS PrivateLink)

You can use AWS PrivateLink to create a private connection between your VPC and Amazon EventBridge Scheduler. You can access EventBridge Scheduler as if it were in your VPC, without the use of an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. Instances in your VPC don't need public IP addresses to access EventBridge Scheduler.

You establish this private connection by creating an *interface endpoint*, powered by AWS PrivateLink. We create an endpoint network interface in each subnet that you enable for the interface endpoint. These are requester-managed network interfaces that serve as the entry point for traffic destined for EventBridge Scheduler.

For more information, see [Access AWS services through AWS PrivateLink](#) in the *AWS PrivateLink Guide*.

Considerations for EventBridge Scheduler

Before you set up an interface endpoint for EventBridge Scheduler, review [Considerations](#) in the *AWS PrivateLink Guide*.

EventBridge Scheduler supports making calls to all of its API actions through the interface endpoint.

Create an interface endpoint for EventBridge Scheduler

You can create an interface endpoint for EventBridge Scheduler using either the Amazon VPC console or the AWS Command Line Interface (AWS CLI). For more information, see [Create an interface endpoint](#) in the *AWS PrivateLink Guide*.

Create an interface endpoint for EventBridge Scheduler using the following service name:

```
com.amazonaws.region.scheduler
```

If you enable private DNS for the interface endpoint, you can make API requests to EventBridge Scheduler using its default Regional DNS name. For example, `scheduler.us-east-1.amazonaws.com`.

Create an endpoint policy for your interface endpoint

An endpoint policy is an IAM resource that you can attach to an interface endpoint. The default endpoint policy allows full access to EventBridge Scheduler through the interface endpoint. To control the access allowed to EventBridge Scheduler from your VPC, attach a custom endpoint policy to the interface endpoint.

An endpoint policy specifies the following information:

- The principals that can perform actions (AWS accounts, IAM users, and IAM roles).
- The actions that can be performed.
- The resources on which the actions can be performed.

For more information, see [Control access to services using endpoint policies](#) in the *AWS PrivateLink Guide*.

Example: VPC endpoint policy for EventBridge Scheduler actions

The following is an example of a custom endpoint policy. When you attach this policy to your interface endpoint, it grants access to the listed EventBridge Scheduler actions for all principals on all resources.

```
{
  "Statement": [
    {
      "Principal": "*",
      "Effect": "Allow",
      "Action": [
        "scheduler:GetSchedule",
        "scheduler:ListSchedules",
        "scheduler:GetScheduleGroup",
        "scheduler:ListScheduleGroups"
      ],
      "Resource": "*"
    }
  ]
}
```


Security in Amazon EventBridge Scheduler

Cloud security at AWS is the highest priority. As an AWS customer, you benefit from data centers and network architectures that are built to meet the requirements of the most security-sensitive organizations.

Security is a shared responsibility between AWS and you. The [shared responsibility model](#) describes this as security *of* the cloud and security *in* the cloud:

- **Security of the cloud** – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely. Third-party auditors regularly test and verify the effectiveness of our security as part of the [AWS Compliance Programs](#). To learn about the compliance programs that apply to Amazon EventBridge Scheduler, see [AWS Services in Scope by Compliance Program](#).
- **Security in the cloud** – Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your company's requirements, and applicable laws and regulations.

This documentation helps you understand how to apply the shared responsibility model when using EventBridge Scheduler. The following topics show you how to configure EventBridge Scheduler to meet your security and compliance objectives. You also learn how to use other AWS services that help you to monitor and secure your EventBridge Scheduler resources.

Topics

- [Managing access to Amazon EventBridge Scheduler](#)
- [Data protection in Amazon EventBridge Scheduler](#)
- [Compliance validation for Amazon EventBridge Scheduler](#)
- [Resilience in Amazon EventBridge Scheduler](#)
- [Infrastructure Security in Amazon EventBridge Scheduler](#)

Managing access to Amazon EventBridge Scheduler

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be *authenticated* (signed in)

and *authorized* (have permissions) to use EventBridge Scheduler resources. IAM is an AWS service that you can use with no additional charge.

Topics

- [Audience](#)
- [Authenticating with identities](#)
- [Managing access using policies](#)
- [How EventBridge Scheduler works with IAM](#)
- [Using identity-based policies in EventBridge Scheduler](#)
- [Confused deputy prevention in EventBridge Scheduler](#)
- [Troubleshooting Amazon EventBridge Scheduler identity and access](#)

Audience

How you use AWS Identity and Access Management (IAM) differs, depending on the work that you do in EventBridge Scheduler.

Service user – If you use the EventBridge Scheduler service to do your job, then your administrator provides you with the credentials and permissions that you need. As you use more EventBridge Scheduler features to do your work, you might need additional permissions. Understanding how access is managed can help you request the right permissions from your administrator. If you cannot access a feature in EventBridge Scheduler, see [Troubleshooting Amazon EventBridge Scheduler identity and access](#).

Service administrator – If you're in charge of EventBridge Scheduler resources at your company, you probably have full access to EventBridge Scheduler. It's your job to determine which EventBridge Scheduler features and resources your service users should access. You must then submit requests to your IAM administrator to change the permissions of your service users. Review the information on this page to understand the basic concepts of IAM. To learn more about how your company can use IAM with EventBridge Scheduler, see [How EventBridge Scheduler works with IAM](#).

IAM administrator – If you're an IAM administrator, you might want to learn details about how you can write policies to manage access to EventBridge Scheduler. To view example EventBridge Scheduler identity-based policies that you can use in IAM, see [Using identity-based policies in EventBridge Scheduler](#).

Authenticating with identities

Authentication is how you sign in to AWS using your identity credentials. You must be *authenticated* (signed in to AWS) as the AWS account root user, as an IAM user, or by assuming an IAM role.

You can sign in to AWS as a federated identity by using credentials provided through an identity source. AWS IAM Identity Center (IAM Identity Center) users, your company's single sign-on authentication, and your Google or Facebook credentials are examples of federated identities. When you sign in as a federated identity, your administrator previously set up identity federation using IAM roles. When you access AWS by using federation, you are indirectly assuming a role.

Depending on the type of user you are, you can sign in to the AWS Management Console or the AWS access portal. For more information about signing in to AWS, see [How to sign in to your AWS account](#) in the *AWS Sign-In User Guide*.

If you access AWS programmatically, AWS provides a software development kit (SDK) and a command line interface (CLI) to cryptographically sign your requests by using your credentials. If you don't use AWS tools, you must sign requests yourself. For more information about using the recommended method to sign requests yourself, see [AWS Signature Version 4 for API requests](#) in the *IAM User Guide*.

Regardless of the authentication method that you use, you might be required to provide additional security information. For example, AWS recommends that you use multi-factor authentication (MFA) to increase the security of your account. To learn more, see [Multi-factor authentication](#) in the *AWS IAM Identity Center User Guide* and [AWS Multi-factor authentication in IAM](#) in the *IAM User Guide*.

AWS account root user

When you create an AWS account, you begin with one sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account *root user* and is accessed by signing in with the email address and password that you used to create the account. We strongly recommend that you don't use the root user for your everyday tasks. Safeguard your root user credentials and use them to perform the tasks that only the root user can perform. For the complete list of tasks that require you to sign in as the root user, see [Tasks that require root user credentials](#) in the *IAM User Guide*.

Federated identity

As a best practice, require human users, including users that require administrator access, to use federation with an identity provider to access AWS services by using temporary credentials.

A *federated identity* is a user from your enterprise user directory, a web identity provider, the AWS Directory Service, the Identity Center directory, or any user that accesses AWS services by using credentials provided through an identity source. When federated identities access AWS accounts, they assume roles, and the roles provide temporary credentials.

For centralized access management, we recommend that you use AWS IAM Identity Center. You can create users and groups in IAM Identity Center, or you can connect and synchronize to a set of users and groups in your own identity source for use across all your AWS accounts and applications. For information about IAM Identity Center, see [What is IAM Identity Center?](#) in the *AWS IAM Identity Center User Guide*.

IAM users and groups

An [IAM user](#) is an identity within your AWS account that has specific permissions for a single person or application. Where possible, we recommend relying on temporary credentials instead of creating IAM users who have long-term credentials such as passwords and access keys. However, if you have specific use cases that require long-term credentials with IAM users, we recommend that you rotate access keys. For more information, see [Rotate access keys regularly for use cases that require long-term credentials](#) in the *IAM User Guide*.

An [IAM group](#) is an identity that specifies a collection of IAM users. You can't sign in as a group. You can use groups to specify permissions for multiple users at a time. Groups make permissions easier to manage for large sets of users. For example, you could have a group named *IAMAdmins* and give that group permissions to administer IAM resources.

Users are different from roles. A user is uniquely associated with one person or application, but a role is intended to be assumable by anyone who needs it. Users have permanent long-term credentials, but roles provide temporary credentials. To learn more, see [Use cases for IAM users](#) in the *IAM User Guide*.

IAM roles

An [IAM role](#) is an identity within your AWS account that has specific permissions. It is similar to an IAM user, but is not associated with a specific person. To temporarily assume an IAM role in the AWS Management Console, you can [switch from a user to an IAM role \(console\)](#). You can assume a

role by calling an AWS CLI or AWS API operation or by using a custom URL. For more information about methods for using roles, see [Methods to assume a role](#) in the *IAM User Guide*.

IAM roles with temporary credentials are useful in the following situations:

- **Federated user access** – To assign permissions to a federated identity, you create a role and define permissions for the role. When a federated identity authenticates, the identity is associated with the role and is granted the permissions that are defined by the role. For information about roles for federation, see [Create a role for a third-party identity provider \(federation\)](#) in the *IAM User Guide*. If you use IAM Identity Center, you configure a permission set. To control what your identities can access after they authenticate, IAM Identity Center correlates the permission set to a role in IAM. For information about permissions sets, see [Permission sets](#) in the *AWS IAM Identity Center User Guide*.
- **Temporary IAM user permissions** – An IAM user or role can assume an IAM role to temporarily take on different permissions for a specific task.
- **Cross-account access** – You can use an IAM role to allow someone (a trusted principal) in a different account to access resources in your account. Roles are the primary way to grant cross-account access. However, with some AWS services, you can attach a policy directly to a resource (instead of using a role as a proxy). To learn the difference between roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.
- **Cross-service access** – Some AWS services use features in other AWS services. For example, when you make a call in a service, it's common for that service to run applications in Amazon EC2 or store objects in Amazon S3. A service might do this using the calling principal's permissions, using a service role, or using a service-linked role.
- **Forward access sessions (FAS)** – When you use an IAM user or role to perform actions in AWS, you are considered a principal. When you use some services, you might perform an action that then initiates another action in a different service. FAS uses the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. FAS requests are only made when a service receives a request that requires interactions with other AWS services or resources to complete. In this case, you must have permissions to perform both actions. For policy details when making FAS requests, see [Forward access sessions](#).
- **Service role** – A service role is an [IAM role](#) that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see [Create a role to delegate permissions to an AWS service](#) in the *IAM User Guide*.

- **Service-linked role** – A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.
- **Applications running on Amazon EC2** – You can use an IAM role to manage temporary credentials for applications that are running on an EC2 instance and making AWS CLI or AWS API requests. This is preferable to storing access keys within the EC2 instance. To assign an AWS role to an EC2 instance and make it available to all of its applications, you create an instance profile that is attached to the instance. An instance profile contains the role and enables programs that are running on the EC2 instance to get temporary credentials. For more information, see [Use an IAM role to grant permissions to applications running on Amazon EC2 instances](#) in the *IAM User Guide*.

Managing access using policies

You control access in AWS by creating policies and attaching them to AWS identities or resources. A policy is an object in AWS that, when associated with an identity or resource, defines their permissions. AWS evaluates these policies when a principal (user, root user, or role session) makes a request. Permissions in the policies determine whether the request is allowed or denied. Most policies are stored in AWS as JSON documents. For more information about the structure and contents of JSON policy documents, see [Overview of JSON policies](#) in the *IAM User Guide*.

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

By default, users and roles have no permissions. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies. The administrator can then add the IAM policies to roles, and users can assume the roles.

IAM policies define permissions for an action regardless of the method that you use to perform the operation. For example, suppose that you have a policy that allows the `iam:GetRole` action. A user with that policy can get role information from the AWS Management Console, the AWS CLI, or the AWS API.

Identity-based policies

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can

perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

Identity-based policies can be further categorized as *inline policies* or *managed policies*. Inline policies are embedded directly into a single user, group, or role. Managed policies are standalone policies that you can attach to multiple users, groups, and roles in your AWS account. Managed policies include AWS managed policies and customer managed policies. To learn how to choose between a managed policy or an inline policy, see [Choose between managed policies and inline policies](#) in the *IAM User Guide*.

Resource-based policies

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must [specify a principal](#) in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services.

Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

Access control lists (ACLs)

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

Amazon S3, AWS WAF, and Amazon VPC are examples of services that support ACLs. To learn more about ACLs, see [Access control list \(ACL\) overview](#) in the *Amazon Simple Storage Service Developer Guide*.

Other policy types

AWS supports additional, less-common policy types. These policy types can set the maximum permissions granted to you by the more common policy types.

- **Permissions boundaries** – A permissions boundary is an advanced feature in which you set the maximum permissions that an identity-based policy can grant to an IAM entity (IAM user

or role). You can set a permissions boundary for an entity. The resulting permissions are the intersection of an entity's identity-based policies and its permissions boundaries. Resource-based policies that specify the user or role in the `Principal` field are not limited by the permissions boundary. An explicit deny in any of these policies overrides the allow. For more information about permissions boundaries, see [Permissions boundaries for IAM entities](#) in the *IAM User Guide*.

- **Service control policies (SCPs)** – SCPs are JSON policies that specify the maximum permissions for an organization or organizational unit (OU) in AWS Organizations. AWS Organizations is a service for grouping and centrally managing multiple AWS accounts that your business owns. If you enable all features in an organization, then you can apply service control policies (SCPs) to any or all of your accounts. The SCP limits permissions for entities in member accounts, including each AWS account root user. For more information about Organizations and SCPs, see [Service control policies](#) in the *AWS Organizations User Guide*.
- **Resource control policies (RCPs)** – RCPs are JSON policies that you can use to set the maximum available permissions for resources in your accounts without updating the IAM policies attached to each resource that you own. The RCP limits permissions for resources in member accounts and can impact the effective permissions for identities, including the AWS account root user, regardless of whether they belong to your organization. For more information about Organizations and RCPs, including a list of AWS services that support RCPs, see [Resource control policies \(RCPs\)](#) in the *AWS Organizations User Guide*.
- **Session policies** – Session policies are advanced policies that you pass as a parameter when you programmatically create a temporary session for a role or federated user. The resulting session's permissions are the intersection of the user or role's identity-based policies and the session policies. Permissions can also come from a resource-based policy. An explicit deny in any of these policies overrides the allow. For more information, see [Session policies](#) in the *IAM User Guide*.

Multiple policy types

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see [Policy evaluation logic](#) in the *IAM User Guide*.

How EventBridge Scheduler works with IAM

Before you use IAM to manage access to EventBridge Scheduler, learn what IAM features are available to use with EventBridge Scheduler.

IAM features you can use with Amazon EventBridge Scheduler

IAM feature	EventBridge Scheduler support
Identity-based policies	Yes
Resource-based policies	No
Policy actions	Yes
Policy resources	Yes
Policy condition keys (service-specific)	Yes
ACLs	No
ABAC (tags in policies)	Partial
Temporary credentials	Yes
Principal permissions	Yes
Service roles	Yes
Service-linked roles	No

To get a high-level view of how EventBridge Scheduler and other AWS services work with most IAM features, see [AWS services that work with IAM](#) in the *IAM User Guide*.

Identity-based policies for EventBridge Scheduler

Supports identity-based policies: Yes

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied. You can't specify the principal in an identity-based policy because it applies to the user or role to which it is attached. To learn about all

of the elements that you can use in a JSON policy, see [IAM JSON policy elements reference](#) in the *IAM User Guide*.

Identity-based policy examples for EventBridge Scheduler

To view examples of EventBridge Scheduler identity-based policies, see [Using identity-based policies in EventBridge Scheduler](#).

Resource-based policies within EventBridge Scheduler

Supports resource-based policies: No

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must [specify a principal](#) in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services.

To enable cross-account access, you can specify an entire account or IAM entities in another account as the principal in a resource-based policy. Adding a cross-account principal to a resource-based policy is only half of establishing the trust relationship. When the principal and the resource are in different AWS accounts, an IAM administrator in the trusted account must also grant the principal entity (user or role) permission to access the resource. They grant permission by attaching an identity-based policy to the entity. However, if a resource-based policy grants access to a principal in the same account, no additional identity-based policy is required. For more information, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Policy actions for EventBridge Scheduler

Supports policy actions: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The **Action** element of a JSON policy describes the actions that you can use to allow or deny access in a policy. Policy actions usually have the same name as the associated AWS API operation. There are some exceptions, such as *permission-only actions* that don't have a matching API operation. There are also some operations that require multiple actions in a policy. These additional actions are called *dependent actions*.

Include actions in a policy to grant permissions to perform the associated operation.

To see a list of EventBridge Scheduler actions, see [Actions defined by Amazon EventBridge Scheduler](#) in the *Service Authorization Reference*.

Policy actions in EventBridge Scheduler use the following prefix before the action:

```
scheduler
```

To specify multiple actions in a single statement, separate them with commas.

```
"Action": [  
    "scheduler:action1",  
    "scheduler:action2"  
]
```

You can specify multiple actions using wildcards (*). For example, to specify all actions that begin with the word `List`, include the following action:

```
"Action": [  
    "scheduler:List*"  
]
```

Policy resources for EventBridge Scheduler

Supports policy resources: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The Resource JSON policy element specifies the object or objects to which the action applies. Statements must include either a Resource or a NotResource element. As a best practice, specify a resource using its [Amazon Resource Name \(ARN\)](#). You can do this for actions that support a specific resource type, known as *resource-level permissions*.

For actions that don't support resource-level permissions, such as listing operations, use a wildcard (*) to indicate that the statement applies to all resources.

```
"Resource": "*"
```

To see a list of EventBridge Scheduler resource types and their ARNs, see [Resources defined by Amazon EventBridge Scheduler](#) in the *Service Authorization Reference*. To learn with which actions you can specify the ARN of each resource, see [Actions defined by Amazon EventBridge Scheduler](#).

To view examples of EventBridge Scheduler identity-based policies, see [Using identity-based policies in EventBridge Scheduler](#).

Policy condition keys for EventBridge Scheduler

Supports service-specific policy condition keys: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The Condition element (or Condition *block*) lets you specify conditions in which a statement is in effect. The Condition element is optional. You can create conditional expressions that use [condition operators](#), such as equals or less than, to match the condition in the policy with values in the request.

If you specify multiple Condition elements in a statement, or multiple keys in a single Condition element, AWS evaluates them using a logical AND operation. If you specify multiple values for a single condition key, AWS evaluates the condition using a logical OR operation. All of the conditions must be met before the statement's permissions are granted.

You can also use placeholder variables when you specify conditions. For example, you can grant an IAM user permission to access a resource only if it is tagged with their IAM user name. For more information, see [IAM policy elements: variables and tags](#) in the *IAM User Guide*.

AWS supports global condition keys and service-specific condition keys. To see all AWS global condition keys, see [AWS global condition context keys](#) in the *IAM User Guide*.

To see a list of EventBridge Scheduler condition keys, see [Condition keys for Amazon EventBridge Scheduler](#) in the *Service Authorization Reference*. To learn with which actions and resources you can use a condition key, see [Actions defined by Amazon EventBridge Scheduler](#).

To view examples of EventBridge Scheduler identity-based policies, see [Using identity-based policies in EventBridge Scheduler](#).

ACLs in EventBridge Scheduler

Supports ACLs: No

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

ABAC with EventBridge Scheduler

Supports ABAC (tags in policies): Partial

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes. In AWS, these attributes are called *tags*. You can attach tags to IAM entities (users or roles) and to many AWS resources. Tagging entities and resources is the first step of ABAC. Then you design ABAC policies to allow operations when the principal's tag matches the tag on the resource that they are trying to access.

ABAC is helpful in environments that are growing rapidly and helps with situations where policy management becomes cumbersome.

To control access based on tags, you provide tag information in the [condition element](#) of a policy using the `aws:ResourceTag/key-name`, `aws:RequestTag/key-name`, or `aws:TagKeys` condition keys.

If a service supports all three condition keys for every resource type, then the value is **Yes** for the service. If a service supports all three condition keys for only some resource types, then the value is **Partial**.

For more information about ABAC, see [Define permissions with ABAC authorization](#) in the *IAM User Guide*. To view a tutorial with steps for setting up ABAC, see [Use attribute-based access control \(ABAC\)](#) in the *IAM User Guide*.

Using temporary credentials with EventBridge Scheduler

Supports temporary credentials: Yes

Some AWS services don't work when you sign in using temporary credentials. For additional information, including which AWS services work with temporary credentials, see [AWS services that work with IAM](#) in the *IAM User Guide*.

You are using temporary credentials if you sign in to the AWS Management Console using any method except a user name and password. For example, when you access AWS using your company's single sign-on (SSO) link, that process automatically creates temporary credentials. You also automatically create temporary credentials when you sign in to the console as a user and then

switch roles. For more information about switching roles, see [Switch from a user to an IAM role \(console\)](#) in the *IAM User Guide*.

You can manually create temporary credentials using the AWS CLI or AWS API. You can then use those temporary credentials to access AWS. AWS recommends that you dynamically generate temporary credentials instead of using long-term access keys. For more information, see [Temporary security credentials in IAM](#).

Cross-service principal permissions for EventBridge Scheduler

Supports forward access sessions (FAS): Yes

When you use an IAM user or role to perform actions in AWS, you are considered a principal. When you use some services, you might perform an action that then initiates another action in a different service. FAS uses the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. FAS requests are only made when a service receives a request that requires interactions with other AWS services or resources to complete. In this case, you must have permissions to perform both actions. For policy details when making FAS requests, see [Forward access sessions](#).

Service roles for EventBridge Scheduler

Supports service roles: Yes

A service role is an [IAM role](#) that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see [Create a role to delegate permissions to an AWS service](#) in the *IAM User Guide*.

Warning

Changing the permissions for a service role might break EventBridge Scheduler functionality. Edit service roles only when EventBridge Scheduler provides guidance to do so.

Service-linked roles for EventBridge Scheduler

Supports service-linked roles: No

A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS

account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.

For details about creating or managing service-linked roles, see [AWS services that work with IAM](#). Find a service in the table that includes a Yes in the **Service-linked role** column. Choose the **Yes** link to view the service-linked role documentation for that service.

Using identity-based policies in EventBridge Scheduler

By default, users and roles don't have permission to create or modify EventBridge Scheduler resources. They also can't perform tasks by using the AWS Management Console, AWS Command Line Interface (AWS CLI), or AWS API. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies. The administrator can then add the IAM policies to roles, and users can assume the roles.

To learn how to create an IAM identity-based policy by using these example JSON policy documents, see [Create IAM policies \(console\)](#) in the *IAM User Guide*.

For details about actions and resource types defined by EventBridge Scheduler, including the format of the ARNs for each of the resource types, see [Actions, resources, and condition keys for Amazon EventBridge Scheduler](#) in the *Service Authorization Reference*.

Topics

- [Policy best practices](#)
- [EventBridge Scheduler permissions](#)
- [AWS managed policies for EventBridge Scheduler](#)
- [Customer managed policies for EventBridge Scheduler](#)
- [AWS managed policy updates](#)

Policy best practices

Identity-based policies determine whether someone can create, access, or delete EventBridge Scheduler resources in your account. These actions can incur costs for your AWS account. When you create or edit identity-based policies, follow these guidelines and recommendations:

- **Get started with AWS managed policies and move toward least-privilege permissions** – To get started granting permissions to your users and workloads, use the *AWS managed policies* that grant permissions for many common use cases. They are available in your AWS account. We

recommend that you reduce permissions further by defining AWS customer managed policies that are specific to your use cases. For more information, see [AWS managed policies](#) or [AWS managed policies for job functions](#) in the *IAM User Guide*.

- **Apply least-privilege permissions** – When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as *least-privilege permissions*. For more information about using IAM to apply permissions, see [Policies and permissions in IAM](#) in the *IAM User Guide*.
- **Use conditions in IAM policies to further restrict access** – You can add a condition to your policies to limit access to actions and resources. For example, you can write a policy condition to specify that all requests must be sent using SSL. You can also use conditions to grant access to service actions if they are used through a specific AWS service, such as AWS CloudFormation. For more information, see [IAM JSON policy elements: Condition](#) in the *IAM User Guide*.
- **Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions** – IAM Access Analyzer validates new and existing policies so that the policies adhere to the IAM policy language (JSON) and IAM best practices. IAM Access Analyzer provides more than 100 policy checks and actionable recommendations to help you author secure and functional policies. For more information, see [Validate policies with IAM Access Analyzer](#) in the *IAM User Guide*.
- **Require multi-factor authentication (MFA)** – If you have a scenario that requires IAM users or a root user in your AWS account, turn on MFA for additional security. To require MFA when API operations are called, add MFA conditions to your policies. For more information, see [Secure API access with MFA](#) in the *IAM User Guide*.

For more information about best practices in IAM, see [Security best practices in IAM](#) in the *IAM User Guide*.

EventBridge Scheduler permissions

In order for an IAM principal (user, group, or role) to create schedules in EventBridge Scheduler and access EventBridge Scheduler resources via the console or the API, the principal must have a set of permissions added to their permission policy. You can configure these permissions depending on the principal's job function. For example, a user, or role, that only uses the EventBridge Scheduler console to view a list of existing schedules does not need to have the permissions required to call the `CreateSchedule` API operation. We recommend tailoring your identity-based permissions to provide only the least privileged access.

The following list shows EventBridge Scheduler's resources, and their corresponding supported actions.

- **Schedule**

- `scheduler:ListSchedules`
- `scheduler:GetSchedule`
- `scheduler>CreateSchedule`
- `scheduler:UpdateSchedule`
- `scheduler>DeleteSchedule`

- **Schedule group**

- `scheduler:ListScheduleGroups`
- `scheduler:GetScheduleGroup`
- `scheduler>CreateScheduleGroup`
- `scheduler>DeleteScheduleGroup`
- `scheduler:ListTagsForResource`
- `scheduler:TagResource`
- `scheduler:UntagResource`

You can use EventBridge Scheduler permissions to create your own customer managed policies to use with EventBridge Scheduler. You can also use the AWS managed policies described in the following section to grant the necessary permissions for common use cases without having to manage your own policies.

AWS managed policies for EventBridge Scheduler

AWS addresses many common use cases by providing standalone IAM policies that AWS creates, and administers. *Managed*, or predefined, policies grant the necessary permissions for common use cases, so you don't need to investigate what permissions are needed. For more information, see [AWS managed policies](#) in the *IAM User Guide*. The following AWS managed policies that you can attach to users in your account are specific to EventBridge Scheduler:

- [the section called "AmazonEventBridgeSchedulerFullAccess"](#) – Grants full access to EventBridge Scheduler using the console and the API.
- [the section called "AmazonEventBridgeSchedulerReadOnlyAccess"](#) – Grants read-only access to EventBridge Scheduler.

AmazonEventBridgeSchedulerFullAccess

The AmazonEventBridgeSchedulerFullAccess managed policy grants permissions to use all EventBridge Scheduler actions for schedules, and schedule groups.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "scheduler:*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::*:role/*",
      "Condition": {
        "StringLike": {
          "iam:PassedToService": "scheduler.amazonaws.com"
        }
      }
    }
  ]
}
```

AmazonEventBridgeSchedulerReadOnlyAccess

The AmazonEventBridgeSchedulerReadOnlyAccess managed policy grants read-only permissions to view details about your schedules and schedule groups.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "scheduler:ListSchedules",
        "scheduler:ListScheduleGroups",
        "scheduler:GetSchedule",
        "scheduler:GetScheduleGroup",
        "scheduler:ListTagsForResource"
      ]
    }
  ]
}
```

```
    ],
    "Resource": "*"
  }
]
}
```

Customer managed policies for EventBridge Scheduler

Use the following examples to create your own customer managed policies for EventBridge Scheduler. [Customer managed policies](#) give let you grant permissions only for the actions and resources required for applications and users in your team according to a principal's job function.

Topics

- [Example: CreateSchedule](#)
- [Example: GetSchedule](#)
- [Example: UpdateSchedule](#)
- [Example: DeleteScheduleGroup](#)

Example: CreateSchedule

When you create a new schedule, you choose whether to encrypt your data on EventBridge Scheduler using an [AWS owned key](#), or a [customer managed key](#).

The following policy allows a principal to create a schedule and apply encryption using an AWS owned key. With an AWS owned key, AWS manages resources on AWS Key Management Service (AWS KMS) for you so you don't need additional permissions to interact with AWS KMS.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "scheduler:CreateSchedule"
      ],
      "Effect": "Allow",
      "Resource": [
```

```

        "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-
schedule-name"
    ],
    },
    {
        "Effect": "Allow",
        "Action": "iam:PassRole",
        "Resource": "arn:aws:iam::123456789012:role/*",
        "Condition": {
            "StringLike": {
                "iam:PassedToService": "scheduler.amazonaws.com"
            }
        }
    }
}

```

Use the following policy to allow a principal create a schedule and use a AWS KMS customer managed key for encryption. To use a customer managed key, a principal must have permission to access the AWS KMS resources in your account. This policy grants access to a single specified KMS key to be used to encrypt data on EventBridge Scheduler. Alternatively, you can use a wildcard (*) character to grant access to all keys in an account, or a subset that matches a given name pattern.

```

{
    "Version": "2012-10-17"
    "Statement":
    [
        {
            "Action":
            [
                "scheduler:CreateSchedule"
            ],
            "Effect": "Allow",
            "Resource":
            [
                "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-
schedule-name"
            ]
        },
        {
            "Action":
            [
                "kms:DescribeKey",

```

```

        "kms:GenerateDataKey",
        "kms:Decrypt"
    ],
    "Effect": "Allow",
    "Resource": [
        "arn:aws:kms:us-west-2:123456789012:key/my-key-id"
    ],
    "Conditions": {
        "StringLike": {
            "kms:ViaService": "scheduler.amazonaws.com",
            "kms:EncryptionContext:aws:scheduler:schedule:arn":
"arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-schedule-name"
        }
    }
    {
        "Effect": "Allow",
        "Action": "iam:PassRole",
        "Resource": "arn:aws:iam::123456789012:role/*",
        "Condition": {
            "StringLike": {
                "iam:PassedToService": "scheduler.amazonaws.com"
            }
        }
    }
}

```

Example: GetSchedule

Use the following policy to allow a principal to get information about a schedule.

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "scheduler:GetSchedule"
            ],
            "Effect": "Allow",
            "Resource": [

```

```

        "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-
schedule-name"
    ]
}

```

Example: UpdateSchedule

Use the following policies to allow a principal to update a schedule by calling the `scheduler:UpdateSchedule` action. Similar to `CreateSchedule`, the policy depends on whether the schedule uses a AWS KMS AWS owned key or a customer managed key for encryption. For a schedule configured with an AWS owned key, use the following policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "scheduler:UpdateSchedule"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-
schedule-name"
      ]
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::123456789012:role/*",
      "Condition": {
        "StringLike": {
          "iam:PassedToService": "scheduler.amazonaws.com"
        }
      }
    }
  ]
}

```

For a schedule configured with a customer managed key, use the following policy. This policy includes additional permissions that allow a principal to access AWS KMS resources in your account:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "scheduler:UpdateSchedule"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-schedule-name"
      ],
    },
    {
      "Action": [
        "kms:DescribeKey",
        "kms:GenerateDataKey",
        "kms:Decrypt"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:kms:us-west-2:123456789012:key/my-key-id"
      ],
      "Conditions": {
        "StringLike": {
          "kms:ViaService": "scheduler.amazonaws.com",
          "kms:EncryptionContext:aws:scheduler:schedule:arn":
            "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-schedule-name"
        }
      }
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::123456789012:role/*",
      "Condition": {
        "StringLike": {

```

```

        "iam:PassedToService": "scheduler.amazonaws.com"
      }
    }
  ]
}

```

Example: DeleteScheduleGroup

Use the following policy to allow a principal to delete a schedule group. When you delete a group, you also delete the schedules associated with that group. The principal that deletes the group must have permission to also delete the schedules associated with that group. This policy grants a principal permission to call the `scheduler:DeleteScheduleGroup` action on the specified schedule groups, as well as all the schedules in the group:

Note

EventBridge Scheduler does not support specifying resource level permissions for individual schedules. For example, the following statement is invalid and should not be included in your policy:

```
"Resource": "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/my-schedule-name"
```

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "scheduler:DeleteSchedule",
      "Resource": "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group/*"
    },
    {
      "Effect": "Allow",
      "Action": "scheduler:DeleteScheduleGroup",
      "Resource": "arn:aws:scheduler:us-west-2:123456789012:schedule/my-group"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::123456789012:role/*",

```



```

        "Condition": {
          "StringLike": {
            "iam:PassedToService": "scheduler.amazonaws.com"
          }
        }
      }
    ]
  }
}

```

AWS managed policy updates

Change	Description	Date
the section called "AmazonEventBridgeSchedulerFullAccess" – New managed policy	EventBridge Scheduler adds support for a new managed policy that grants users full access to all resources , including schedules, and schedule groups.	November 10, 2022
the section called "AmazonEventBridgeSchedulerReadOnlyAccess" – New managed policy	EventBridge Scheduler adds support for a new managed policy that grants users read-only access to all resources , including schedules, and schedule groups.	November 10, 2022
EventBridge Scheduler started tracking changes	EventBridge Scheduler started tracking changes for its AWS managed policies.	November 10, 2022

Confused deputy prevention in EventBridge Scheduler

The confused deputy problem is a security issue where an entity that doesn't have permission to perform an action can coerce a more-privileged entity to perform the action. In AWS, cross-service impersonation can result in the confused deputy problem. Cross-service impersonation can occur when one service (the *calling service*) calls another service (the *called service*). The calling service can be manipulated to use its permissions to act on another customer's resources in a way it should

not otherwise have permission to access. To prevent this, AWS provides tools that help you protect your data for all services with service principals that have been given access to resources in your account.

We recommend using the [aws:SourceArn](#) and [aws:SourceAccount](#) global condition context keys in your schedule execution role to limit the permissions that EventBridge Scheduler gives another service to access the resource. Use `aws:SourceArn` if you want only one resource to be associated with the cross-service access. Use `aws:SourceAccount` if you want to allow any resource in that account to be associated with the cross-service use.

The most effective way to protect against the confused deputy problem is to use the `aws:SourceArn` global condition context key with the full ARN of the resource. The following condition is scoped to an individual schedule group:

```
arn:aws:scheduler:*:123456789012:schedule-group/your-schedule-group
```

If you don't know the full ARN of the resource or if you are specifying multiple resources, use the `aws:SourceArn` global context condition key with wildcard characters (*) for the unknown portions of the ARN. For example: `arn:aws:scheduler:*:123456789012:schedule-group/*`.

The value of `aws:SourceArn` must be your EventBridge Scheduler schedule group ARN to which you want to scope this condition.

Important

Do not scope the `aws:SourceArn` statement to a specific schedule or a schedule name prefix. The ARN you specify must be a schedule group.

The following example shows how you can use the `aws:SourceArn` and `aws:SourceAccount` global condition context keys in your execution role trust policy to prevent the confused deputy problem:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "scheduler.amazonaws.com"
```

```
    },
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": {
        "aws:SourceAccount": "123456789012",
        "aws:SourceArn": "arn:aws:scheduler:us-west-2:123456789012:schedule-group/your-schedule-group"
      }
    }
  }
]
```

Troubleshooting Amazon EventBridge Scheduler identity and access

Use the following information to help you diagnose and fix common issues that you might encounter when working with EventBridge Scheduler and IAM.

Topics

- [I am not authorized to perform an action in EventBridge Scheduler](#)
- [I am not authorized to perform iam:PassRole](#)
- [I want to allow people outside of my AWS account to access my EventBridge Scheduler resources](#)

I am not authorized to perform an action in EventBridge Scheduler

If you receive an error that you're not authorized to perform an action, your policies must be updated to allow you to perform the action.

The following example error occurs when the mateojackson IAM user tries to use the console to view details about a fictional *my-example-widget* resource but does not have the fictional scheduler:*GetWidget* permissions.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
scheduler:GetWidget on resource: my-example-widget
```

In this case, Mateo's policy must be updated to allow him to access the *my-example-widget* resource using the scheduler:*GetWidget* action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

I am not authorized to perform iam:PassRole

If you receive an error that you're not authorized to perform the `iam:PassRole` action, your policies must be updated to allow you to pass a role to EventBridge Scheduler.

Some AWS services allow you to pass an existing role to that service instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in EventBridge Scheduler. However, the action requires the service to have permissions that are granted by a service role. Mary does not have permissions to pass the role to the service.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

In this case, Mary's policies must be updated to allow her to perform the `iam:PassRole` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

I want to allow people outside of my AWS account to access my EventBridge Scheduler resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether EventBridge Scheduler supports these features, see [How EventBridge Scheduler works with IAM](#).
- To learn how to provide access to your resources across AWS accounts that you own, see [Providing access to an IAM user in another AWS account that you own](#) in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see [Providing access to AWS accounts owned by third parties](#) in the *IAM User Guide*.

- To learn how to provide access through identity federation, see [Providing access to externally authenticated users \(identity federation\)](#) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Data protection in Amazon EventBridge Scheduler

The AWS [shared responsibility model](#) applies to data protection in Amazon EventBridge Scheduler. As described in this model, AWS is responsible for protecting the global infrastructure that runs all of the AWS Cloud. You are responsible for maintaining control over your content that is hosted on this infrastructure. You are also responsible for the security configuration and management tasks for the AWS services that you use. For more information about data privacy, see the [Data Privacy FAQ](#). For information about data protection in Europe, see the [AWS Shared Responsibility Model and GDPR](#) blog post on the *AWS Security Blog*.

For data protection purposes, we recommend that you protect AWS account credentials and set up individual users with AWS IAM Identity Center or AWS Identity and Access Management (IAM). That way, each user is given only the permissions necessary to fulfill their job duties. We also recommend that you secure your data in the following ways:

- Use multi-factor authentication (MFA) with each account.
- Use SSL/TLS to communicate with AWS resources. We require TLS 1.2 and recommend TLS 1.3.
- Set up API and user activity logging with AWS CloudTrail. For information about using CloudTrail trails to capture AWS activities, see [Working with CloudTrail trails](#) in the *AWS CloudTrail User Guide*.
- Use AWS encryption solutions, along with all default security controls within AWS services.
- Use advanced managed security services such as Amazon Macie, which assists in discovering and securing sensitive data that is stored in Amazon S3.
- If you require FIPS 140-3 validated cryptographic modules when accessing AWS through a command line interface or an API, use a FIPS endpoint. For more information about the available FIPS endpoints, see [Federal Information Processing Standard \(FIPS\) 140-3](#).

We strongly recommend that you never put confidential or sensitive information, such as your customers' email addresses, into tags or free-form text fields such as a **Name** field. This includes when you work with EventBridge Scheduler or other AWS services using the console, API, AWS

CLI, or AWS SDKs. Any data that you enter into tags or free-form text fields used for names may be used for billing or diagnostic logs. If you provide a URL to an external server, we strongly recommend that you do not include credentials information in the URL to validate your request to that server.

Encryption at rest in EventBridge Scheduler

This section describes how Amazon EventBridge Scheduler encrypts and decrypts your data at rest. Data at rest is data stored in EventBridge Scheduler and the service's underlying components. EventBridge Scheduler integrates with AWS Key Management Service (AWS KMS) to encrypt and decrypt your data using an [AWS KMS key](#). EventBridge Scheduler supports two types of KMS keys: [AWS owned keys](#), and [customer managed keys](#).

Note

EventBridge Scheduler only supports using [symmetric](#) encryption KMS keys.

AWS owned keys are KMS keys that an AWS service owns and manages for use in multiple AWS accounts. Although the AWS owned keys EventBridge Scheduler uses are not stored in your AWS account, EventBridge Scheduler uses them to protect your data and resources. By default, EventBridge Scheduler encrypts and decrypts all of your data using an AWS owned key. You do not need to manage your AWS owned key or its access policy. You do not incur any fees when EventBridge Scheduler uses AWS owned keys to protect your data, and their usage does not count as part of your AWS KMS quotas in your account.

Customer managed keys are KMS keys stored in your AWS account that you create, own, and manage. If your specific use case requires that you control and audit the encryption keys that protect your data on EventBridge Scheduler, you can use a customer managed key. If you choose a customer managed key, you must manage your key policy. Customer managed keys incur a monthly fee and a fee for use in excess of the free tier. Using a customer managed key also counts as part of your [AWS KMS quota](#). For more information on pricing, see [AWS Key Management Service pricing](#).

Topics

- [Encryption artifacts](#)
- [Managing KMS keys](#)
- [CloudTrail event example](#)

Encryption artifacts

The following table describes the different types of data that EventBridge Scheduler encrypts at rest, and which type of KMS key it supports for each category.

Data type	Description	AWS owned key	customer managed key
Payload (up to 256KB)	The data that you specify in the schedule's <code>TargetInput</code> parameter when you configure the schedule to be delivered to the target.	Supported	Supported
Identifier and state	The unique name and the state (enable, disable) of the schedule.	Supported	Not supported
Scheduling configuration	The scheduling expression, such as the rate or cron expression for recurring schedules, and the timestamp for one-time invocations, as well as the schedule's start date, end date, and time zone.	Supported	Not supported
Target configuration	The target's Amazon Resource Name (ARN), and other	Supported	Not supported

Data type	Description	AWS owned key	customer managed key
	target related configuration details.		
Invocation and failure behavior configuration	Flexible time window configuration, the schedule's retry policy, and the dead-letter queue details used for failed deliveries.	Supported	Not supported

EventBridge Scheduler uses your customer managed keys only when encrypting and decrypting the target payload, as described in the previous table. If you choose to use a customer managed key, EventBridge Scheduler encrypts and decrypts the payload twice: once using the default AWS owned key, and another time using the customer managed key that you specify. For all other data types, EventBridge Scheduler only uses the default AWS owned key to protect your data at rest.

Use the following [the section called “Managing KMS keys”](#) section to learn how you must manage your IAM resources and key policies in order to use a customer managed key with EventBridge Scheduler.

Managing KMS keys

You can optionally provide a customer managed key to encrypt and decrypt the payload that your schedule delivers to its target. EventBridge Scheduler encrypts and decrypts your payload up to 256KB of data. Using a customer managed key incurs a monthly fee and a fee in excess of the free tier. Using a customer managed key counts as part of your [AWS KMS quota](#). For more information on pricing, see [AWS Key Management Service pricing](#)

EventBridge Scheduler uses IAM permissions associated with the principal that creates a schedule to encrypt your data. This means you must attach the required AWS KMS related permissions to the user, or role, that calls the EventBridge Scheduler API. In addition, EventBridge Scheduler uses resource-based policies to decrypt your data. This means that the execution role associated with your schedule must also have the required AWS KMS related permissions to call the AWS KMS API when decrypting data.

Note

EventBridge Scheduler does not support using [grants](#) for temporary permissions.

Use the following section to learn how you can manage your AWS KMS [key policy](#) and required IAM permissions to use a customer managed key on EventBridge Scheduler.

Topics

- [Add IAM permissions](#)
- [Manage the key policy](#)

Add IAM permissions

To use a customer managed key, you must add the following permissions to the identity-based IAM principal that creates a schedule, as well as the execution role you associate with the schedule.

Identity-based permissions for customer managed keys

You must add the following AWS KMS actions to the permission policy associated with any principal (users, groups, or roles) that calls the EventBridge Scheduler API when creating a schedule.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "scheduler:*",

        # Required to pass the execution role
        "iam:PassRole",

        "kms:DescribeKey",
        "kms:GenerateDataKey",
        "kms:Decrypt"
      ],
      "Resource": "*",
      "Effect": "Allow"
    },
  ],
}
```

```
]
}
```

- **kms:DescribeKey** – Required in order to validate that the key you provide is a [symmetric](#) encryption KMS key.
- **kms:GenerateDataKey** – Required in order to generate the data key that EventBridge Scheduler uses to perform client side encryption.
- **kms:Decrypt** – Required decrypt the encrypted data key that EventBridge Scheduler stores together with your encrypted data.

Execution role permissions for customer managed keys

You must add the following action to your schedule's execution role permissions policy to provide access to EventBridge Scheduler to call the AWS KMS API when decrypting your data.

```
{
  "Version": "2012-10-17",
  "Statement" : [
    {
      "Sid" : "Allow EventBridge Scheduler to decrypt data using a customer managed
key",
      "Effect" : "Allow",
      "Action" : [
        "kms:Decrypt"
      ],
      "Resource": "arn:aws:kms:your-region:123456789012:key/your-key-id"
    }
  ]
}
```

- **kms:Decrypt** – Required decrypt the encrypted data key that EventBridge Scheduler stores together with your encrypted data.

If you use the EventBridge Scheduler console to create a new execution role when you create a new schedule, EventBridge Scheduler will automatically attach the required permission to your execution role. However, if you choose an existing execution role, you must add the required permissions to the role to be able to use your customer managed keys.

Manage the key policy

When you create a customer managed key using AWS KMS, by default, your key has the following key policy to provide access to your schedules' execution roles.

```
{
  "Id": "key-policy-1",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Provide required IAM Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    }
  ]
}
```

Optionally, you can limit the scope of your key policy to only provide access to the execution role. You might do this if you want to use your customer managed key only with your EventBridge Scheduler resources. Use the following [key policy](#) example to limit which EventBridge Scheduler resources can use your key.

```
{
  "Id": "key-policy-2",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Provide required IAM Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::695325144837:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    },
    {
      "Sid": "Allow use of the key",
      "Effect": "Allow",

```

```

    "Principal": {
      "AWS": "arn:aws:iam::123456789012:role/schedule-execution-role"
    },
    "Action": [
      "kms:Decrypt"
    ],
    "Resource": "*"
  }
]
}

```

CloudTrail event example

AWS CloudTrail captures all API calls *events*. This includes API calls whenever EventBridge Scheduler uses your customer managed key to decrypt your data. The following example shows a CloudTrail event entry that demonstrates EventBridge Scheduler using the `kms:Decrypt` action using a customer managed key.

```

{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "AssumedRole",
    "principalId": "ABCDEABCD1AB12ABABAB0:70abcd123a123a12345a1aa12aa1bc12",
    "arn": "arn:aws:sts::123456789012:assumed-role/execution-role/70abcd123a123a12345a1aa12aa1bc12",
    "accountId": "123456789012",
    "accessKeyId": "ABCDEFGH11JKLMN0P2Q3",
    "sessionContext": {
      "sessionIssuer": {
        "type": "Role",
        "principalId": "ABCDEABCD1AB12ABABAB0",
        "arn": "arn:aws:iam::123456789012:role/execution-role",
        "accountId": "123456789012",
        "userName": "execution-role"
      },
      "webIdFederationData": {},
      "attributes": {
        "creationDate": "2022-10-31T21:03:15Z",
        "mfaAuthenticated": "false"
      }
    }
  },
  "eventTime": "2022-10-31T21:03:15Z",

```

```

    "eventSource": "kms.amazonaws.com",
    "eventName": "Decrypt",
    "awsRegion": "eu-north-1",
    "sourceIPAddress": "13.50.87.173",
    "userAgent": "aws-sdk-java/2.17.295 Linux/4.14.291-218.527.amzn2.x86_64 OpenJDK_64-
    Bit_Server_VM/11.0.17+9-LTS Java/11.0.17 kotlin/1.3.72-release-468 (1.3.72) vendor/
    Amazon.com_Inc. md/internal exec-env/AWS_ECS_FARGATE io/sync http/Apache cfg/retry-
    mode/standard AwsCrypto/2.4.0",
    "requestParameters": {
        "keyId": "arn:aws:kms:us-west-2:123456789012:key/2321abab-2110-12ab-a123-
        a2b34c5abc67",
        "encryptionAlgorithm": "SYMMETRIC_DEFAULT",
        "encryptionContext": {
            "aws:scheduler:schedule:arn": "arn:aws:scheduler:us-
            west-2:123456789012:schedule/default/execution-role"
        }
    },
    "responseElements": null,
    "requestID": "request-id",
    "eventID": "event-id",
    "readOnly": true,
    "resources": [
        {
            "accountId": "123456789012",
            "type": "AWS::KMS::Key",
            "ARN": "arn:aws:kms:us-west-2:123456789012:key/2321abab-2110-12ab-a123-
            a2b34c5abc67"
        }
    ],
    "eventType": "AwsApiCall",
    "managementEvent": true,
    "recipientAccountId": "123456789012",
    "eventCategory": "Management",
    "tlsDetails": {
        "tlsVersion": "TLSv1.3",
        "cipherSuite": "TLS_AES_256_GCM_SHA384",
        "clientProvidedHostHeader": "kms.us-west-2.amazonaws.com"
    }
}

```

Encryption in transit in EventBridge Scheduler

EventBridge Scheduler encrypts your data in transit as it travels the network. Transport Layer Security (TLS) encrypts your data when you call any EventBridge Scheduler API operations, as well as when EventBridge Scheduler calls any target APIs when it invokes your schedule. By default, EventBridge Scheduler uses TLS 1.2 when encrypting your data in transit. You do not need to configure encryption in transit, and you cannot choose a different TLS version when using EventBridge Scheduler.

Using the EventBridge Scheduler API – When you perform an API operation, such as `CreateSchedule`, EventBridge Scheduler encrypts the entire HTTP request, including the request body and headers. EventBridge Scheduler also encrypts the entire response object that you receive from our APIs.

Using target APIs – When EventBridge Scheduler invokes your schedule, it calls the target API that you specified when you created the schedule. When delivering an event to a target, EventBridge Scheduler encrypts the entire request, including the request body and all headers, as well as the response it receives from the target.

Compliance validation for Amazon EventBridge Scheduler

To learn whether an AWS service is within the scope of specific compliance programs, see [AWS services in Scope by Compliance Program](#) and choose the compliance program that you are interested in. For general information, see [AWS Compliance Programs](#).

You can download third-party audit reports using AWS Artifact. For more information, see [Downloading Reports in AWS Artifact](#).

Your compliance responsibility when using AWS services is determined by the sensitivity of your data, your company's compliance objectives, and applicable laws and regulations. AWS provides the following resources to help with compliance:

- [Security Compliance & Governance](#) – These solution implementation guides discuss architectural considerations and provide steps for deploying security and compliance features.
- [HIPAA Eligible Services Reference](#) – Lists HIPAA eligible services. Not all AWS services are HIPAA eligible.
- [AWS Compliance Resources](#) – This collection of workbooks and guides might apply to your industry and location.

- [AWS Customer Compliance Guides](#) – Understand the shared responsibility model through the lens of compliance. The guides summarize the best practices for securing AWS services and map the guidance to security controls across multiple frameworks (including National Institute of Standards and Technology (NIST), Payment Card Industry Security Standards Council (PCI), and International Organization for Standardization (ISO)).
- [Evaluating Resources with Rules](#) in the *AWS Config Developer Guide* – The AWS Config service assesses how well your resource configurations comply with internal practices, industry guidelines, and regulations.
- [AWS Security Hub](#) – This AWS service provides a comprehensive view of your security state within AWS. Security Hub uses security controls to evaluate your AWS resources and to check your compliance against security industry standards and best practices. For a list of supported services and controls, see [Security Hub controls reference](#).
- [Amazon GuardDuty](#) – This AWS service detects potential threats to your AWS accounts, workloads, containers, and data by monitoring your environment for suspicious and malicious activities. GuardDuty can help you address various compliance requirements, like PCI DSS, by meeting intrusion detection requirements mandated by certain compliance frameworks.
- [AWS Audit Manager](#) – This AWS service helps you continuously audit your AWS usage to simplify how you manage risk and compliance with regulations and industry standards.

Resilience in Amazon EventBridge Scheduler

The AWS global infrastructure is built around AWS Regions and Availability Zones. AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected with low-latency, high-throughput, and highly redundant networking. With Availability Zones, you can design and operate applications and databases that automatically fail over between zones without interruption. Availability Zones are more highly available, fault tolerant, and scalable than traditional single or multiple data center infrastructures.

For more information about AWS Regions and Availability Zones, see [AWS Global Infrastructure](#).

In addition to the AWS global infrastructure, EventBridge Scheduler offers several features to help support your data resiliency and backup needs.

Infrastructure Security in Amazon EventBridge Scheduler

As a managed service, Amazon EventBridge Scheduler is protected by AWS global network security. For information about AWS security services and how AWS protects infrastructure, see [AWS Cloud Security](#). To design your AWS environment using the best practices for infrastructure security, see [Infrastructure Protection](#) in *Security Pillar AWS Well-Architected Framework*.

You use AWS published API calls to access EventBridge Scheduler through the network. Clients must support the following:

- Transport Layer Security (TLS). We require TLS 1.2 and recommend TLS 1.3.
- Cipher suites with perfect forward secrecy (PFS) such as DHE (Ephemeral Diffie-Hellman) or ECDHE (Elliptic Curve Ephemeral Diffie-Hellman). Most modern systems such as Java 7 and later support these modes.

Additionally, requests must be signed by using an access key ID and a secret access key that is associated with an IAM principal. Or you can use the [AWS Security Token Service](#) (AWS STS) to generate temporary security credentials to sign requests.

Monitoring and metrics for Amazon EventBridge Scheduler

Monitoring is an important part of maintaining the reliability, availability, and performance of Amazon EventBridge Scheduler and your other AWS solutions. AWS provides the following monitoring tools to watch EventBridge Scheduler, report when something is wrong, and take automatic actions when appropriate:

- *Amazon CloudWatch* monitors your AWS resources and the applications you run on AWS in real time. You can collect and track metrics, create customized dashboards, and set alarms that notify you or take actions when a specified metric reaches a threshold that you specify. For more information, see the [Amazon CloudWatch User Guide](#).
- *AWS CloudTrail* captures API calls and related events made by or on behalf of your AWS account and delivers the log files to an Amazon S3 bucket that you specify. You can identify which users and accounts called AWS, the source IP address from which the calls were made, and when the calls occurred. For more information, see the [AWS CloudTrail User Guide](#).

Topics

- [Monitoring Amazon EventBridge Scheduler with Amazon CloudWatch](#)
- [Logging Amazon EventBridge Scheduler API calls using AWS CloudTrail](#)

Monitoring Amazon EventBridge Scheduler with Amazon CloudWatch

You can monitor Amazon EventBridge Scheduler using CloudWatch, which collects raw data and processes it into readable, near real-time metrics. EventBridge Scheduler emits a set of metrics for all schedules, and an additional set of metrics for schedules that have an associated dead-letter queue (DLQ). If you [configure a DLQ](#) for your schedule, EventBridge Scheduler publishes additional metrics when your schedule exhausts its retry policy.

These statistics are kept for 15 months, so that you can access historical information and gain a better perspective on why a schedule is failing, and troubleshoot underlying issues. You can also set alarms that watch for certain thresholds, and send notifications or take actions when those thresholds are met. For more information, see the [Amazon CloudWatch User Guide](#).

Topics

- [Terms](#)
- [Dimensions](#)
- [Accessing metrics](#)
- [List of metrics](#)
- [EventBridge Scheduler usage metrics](#)

Terms

Namespace

A namespace is a container for the CloudWatch metrics of an AWS service. For EventBridge Scheduler, the namespace is `AWS/Scheduler`.

CloudWatch metrics

A CloudWatch metric represents a time-ordered set of data points that are specific to CloudWatch.

Dimension

A dimension is a name/value pair that is part of the identity of a metric.

Unit

A statistic has a unit of measure. For EventBridge Scheduler, units include *Count*.

Dimensions

This section describes the CloudWatch dimensions grouping for EventBridge Scheduler metrics in CloudWatch.

Dimension	Description
ScheduleGroup	The group of schedules for which you want to view metrics using CloudWatch. If you have not created any groups yet, EventBridge Scheduler associates your schedules with the default group.

Accessing metrics

This section describes how to access performance metrics in CloudWatch for a specific EventBridge Scheduler schedule.

To view performance metrics for a dimension

1. Open the [Metrics page](#) on the CloudWatch console.
2. Use the AWS Region selector to choose the Region for your schedule
3. Choose the **Scheduler** namespace.
4. In the **All metrics** tab, choose a dimension, for example, **Schedule Group Metrics**. To see metrics for all the schedules you've created in your selected Region, choose **Account Metrics**.
5. Choose a CloudWatch metric for a dimension. For example, **InvocationAttemptCount** or **InvocationDroppedCount**, then choose **Graph search**.
6. Choose the **Graphed metrics** tab to view performance statistics for EventBridge Scheduler metrics.

List of metrics

The following tables list the metrics for all EventBridge Scheduler schedules, as well as additional metrics for schedules for which you've configured a DLQ.

Metrics for all schedules

Namespace	Metric	Unit	Description
AWS/Scheduler	InvocationAttemptCount	Count	Emitted for every invocation attempt. Use this metric to check that EventBridge Scheduler is attempting to invoke your schedules, and to see when invocations approach your account quotas.

Namespace	Metric	Unit	Description
AWS/Scheduler	TargetErrorCount	Count	Emitted when the target returns an exception after EventBridge Scheduler calls the target API. Use this to check when delivery to a target fails.
AWS/Scheduler	TargetErrorThrottledCount	Count	Emitted when target invocation fails due to API throttling by the target. Use this to diagnose delivery failures when the underlying reason is the target API throttling calls made by EventBridge Scheduler

Namespace	Metric	Unit	Description
AWS/Scheduler	InvocationThrottleCount	Count	Emitted when EventBridge Scheduler throttles a target invocation because it exceeds your service quotas set by EventBridge Scheduler. Use this to determine when you have exceeded your invocations throttle limit quota. For more information about service quotas, see Quotas .
AWS/Scheduler	InvocationDroppedCount	Count	Emitted when EventBridge Scheduler stops attempting to invoke the target after a schedule's retry policy has been exhausted. For more information about retry policies, see RetryPolicy in the <i>EventBridge Scheduler API Reference</i> .

Metrics for schedules with a DLQ

Namespace	Metric	Unit	Description
AWS/Scheduler	InvocationsSentToDeadLetterCount	Count	Emitted for every successful delivery to a schedule's DLQ. Use this to determine when events are sent to a DLQ, then check the event delivered to the schedule's DLQ for additional details that help you determine the cause of the failure.
AWS/Scheduler	InvocationsFailedToBeSentToDeadLetterCount	Count	Emitted when EventBridge Scheduler cannot deliver an event to the DLQ. Use these two metrics to determine
AWS/Scheduler	InvocationsFailedToBeSentToDeadLetterCount_<error_code>	Count	

Namespace	Metric	Unit	Description
			the reason why EventBridge Scheduler is unable to send an event to the DLQ, and modify your DLQ configuration to resolve the issue. The following is an example of the <code>InvocationsFailedToBeSentToDeadLetterCount_<error_code></code> metric when the Amazon SQS queue you specify as a DLQ does not exist: <code>InvocationsFailedToBeSentToDeadLetter</code>

Namespace	Metric	Unit	Description
			Count AWS.SimpleQueueService.NonExistentQueue
AWS/Scheduler	InvocationsSentToDeadLetterCount_Truncated_MessageSize Exceeded	Count	Emitted when the payload of the event sent to the DLQ exceeds the maximum size allowed by Amazon SQS, and EventBridge Scheduler truncates the payload you specify in the Input attribute of a schedule.

EventBridge Scheduler usage metrics

CloudWatch collects metrics that track the usage of some AWS resources. These metrics correspond to AWS service quotas. Tracking these metrics can help you proactively manage your quotas. Use the following metrics to determine when you have exceeded your EventBridge Scheduler quotas. For more information about service quotas, see [Quotas](#).

These metrics are contained in the AWS/Usage namespace, rather than AWS/Scheduler, and are collected every minute.

Currently, the only metric name in this namespace that CloudWatch publishes is `CallCount`. This metric is published with the dimensions `Resource`, `Service`, and `Type`. The `Resource` dimension specifies the name of the API operation being tracked.

For example, the `CallCount` metric with the following dimensions indicates the number of times the EventBridge Scheduler `CreateSchedule` API operation has been called in your account:

- "Service": "Scheduler"
- "Type": "API"
- "Resource": "CreateSchedule"

The `CallCount` metric does not have a specified unit. The most useful statistic for the metric is `SUM`, which represents the total operation count for the 1-minute period.

Metrics

Metric	Description		
<code>CallCount</code>	The number of specified operations performed in your account.		

Dimensions

Dimension	Description		
<code>Service</code>	The name of the AWS service containing the resource. For EventBridge Scheduler usage metrics, the value for this dimension is <code>Scheduler</code> .		
<code>Class</code>	The class of resource being tracked. EventBridge Scheduler API usage metrics use this dimension with a value of <code>None</code> .		

Dimension	Description		
Type	The type of resource being tracked. Currently, when the Service dimension is Scheduler , the only valid value for Type is API.		
Resource	The name of the API operation. Valid values include the following: • CreateSchedule • CreateScheduleGroup • DeleteSchedule • DeleteScheduleGroup • GetSchedule • GetScheduleGroup • ListScheduleGroups • ListSchedules • ListTagsForResource • TagResource • UntagResource • UpdateSchedule		

Logging Amazon EventBridge Scheduler API calls using AWS CloudTrail

Amazon EventBridge Scheduler is integrated with AWS CloudTrail, a service that provides a record of actions taken by a user, role, or an AWS service in EventBridge Scheduler. CloudTrail captures all API calls for EventBridge Scheduler as events. The calls captured include calls from the EventBridge Scheduler console and code calls to the EventBridge Scheduler API operations. If you create a trail, you can enable continuous delivery of CloudTrail events to an Amazon S3 bucket, including events for EventBridge Scheduler. If you don't configure a trail, you can still view the most recent events in the CloudTrail console in **Event history**. Using the information collected by CloudTrail, you can

determine the request that was made to EventBridge Scheduler, the IP address from which the request was made, who made the request, when it was made, and additional details.

To learn more about CloudTrail, see the [AWS CloudTrail User Guide](#).

EventBridge Scheduler information in CloudTrail

CloudTrail is enabled on your AWS account when you create the account. When activity occurs in EventBridge Scheduler, that activity is recorded in a CloudTrail event along with other AWS service events in **Event history**. You can view, search, and download recent events in your AWS account. For more information, see [Viewing events with CloudTrail Event history](#).

For an ongoing record of events in your AWS account, including events for EventBridge Scheduler, create a trail. A *trail* enables CloudTrail to deliver log files to an Amazon S3 bucket. By default, when you create a trail in the console, the trail applies to all AWS Regions. The trail logs events from all Regions in the AWS partition and delivers the log files to the Amazon S3 bucket that you specify. Additionally, you can configure other AWS services to further analyze and act upon the event data collected in CloudTrail logs. For more information, see the following:

- [Overview for creating a trail](#)
- [CloudTrail supported services and integrations](#)
- [Configuring Amazon SNS notifications for CloudTrail](#)
- [Receiving CloudTrail log files from multiple regions](#) and [Receiving CloudTrail log files from multiple accounts](#)

All EventBridge Scheduler API actions are logged by CloudTrail and are documented in the [Amazon EventBridge Scheduler API Reference](#). For example, calls to the `CreateSchedule`, `UpdateSchedule` and `DeleteSchedule` actions generate entries in the CloudTrail log files.

Every event or log entry contains information about who generated the request. The identity information helps you determine the following:

- Whether the request was made with root or AWS Identity and Access Management (IAM) user credentials.
- Whether the request was made with temporary security credentials for a role or federated user.
- Whether the request was made by another AWS service.

For more information, see the [CloudTrail userIdentity element](#).

Understanding EventBridge Scheduler log file entries

A trail is a configuration that enables delivery of events as log files to an Amazon S3 bucket that you specify. CloudTrail log files contain one or more log entries. An event represents a single request from any source and includes information about the requested action, the date and time of the action, request parameters, and so on. CloudTrail log files aren't an ordered stack trace of the public API calls, so they don't appear in any specific order.

Quotas for Amazon EventBridge Scheduler

Your AWS account has default quotas, formerly referred to as limits, for each AWS service. Unless otherwise noted, each quota is Region-specific. You can request increases for most quotas, but some cannot be increased.

To view the quotas for EventBridge Scheduler, open the [Service Quotas console](#). In the navigation pane, choose **AWS services**, then select **EventBridge Scheduler**.

To request a quota increase, see [Requesting a quota increase](#) in the *Service Quotas User Guide*. If the quota is not yet available in Service Quotas, use the [limit increase form](#).

Your AWS account has the following quotas related to EventBridge Scheduler.

Name	Default	Adjustable	Description
CreateSchedule request rate	us-east-1: 1,000 us-east-2: 1,000 us-west-2: 1,000 ap-northeast-1: 1,000 ap-south-1: 1,000 ap-southeast-1: 1,000 ap-southeast-2: 1,000 eu-central-1: 1,000 eu-west-1: 1,000 eu-west-2: 1,000	Yes	Maximum CreateSchedule requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval. This is adjustable to thousands of requests per second.

Name	Default	Adjustable	Description
	sa-east-1: 1,000 Each of the other supported Regions: 250		
CreateScheduleGroup request rate	Each supported Region: 10	Yes	Maximum CreateScheduleGroup requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval.

Name	Default	Adjustable	Description
DeleteSchedule request rate	us-east-1: 1,000 us-east-2: 1,000 us-west-2: 1,000 ap-northeast-1: 1,000 ap-south-1: 1,000 ap-southeast-1: 1,000 ap-southeast-2: 1,000 eu-central-1: 1,000 eu-west-1: 1,000 eu-west-2: 1,000 sa-east-1: 1,000 Each of the other supported Regions: 250	Yes	Maximum DeleteSchedule requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval. This is adjustable to thousands of requests per second.

Name	Default	Adjustable	Description
DeleteScheduleGroup request rate	Each supported Region: 10	Yes	Maximum DeleteScheduleGroup requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval.

Name	Default	Adjustable	Description
GetSchedule request rate	us-east-1: 1,000 us-east-2: 1,000 us-west-2: 1,000 ap-northeast-1: 1,000 ap-south-1: 1,000 ap-southeast-1: 1,000 ap-southeast-2: 1,000 eu-central-1: 1,000 eu-west-1: 1,000 eu-west-2: 1,000 sa-east-1: 1,000 Each of the other supported Regions: 250	Yes	Maximum GetSchedule requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval. This is adjustable to thousands of requests per second.

Name	Default	Adjustable	Description
GetScheduleGroup request rate	Each supported Region: 10	Yes	Maximum GetScheduleGroup requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval.

Name	Default	Adjustable	Description
Invocations throttle limit in transactions per second	us-east-1: 1,000 us-east-2: 1,000 us-west-2: 1,000 ap-northeast-1: 1,000 ap-south-1: 1,000 ap-southeast-1: 1,000 ap-southeast-2: 1,000 eu-central-1: 1,000 eu-west-1: 1,000 eu-west-2: 1,000 sa-east-1: 1,000 Each of the other supported Regions: 500	Yes	An invocation is a scheduled payload being delivered to the defined target. After the limit is reached, the invocations are throttled; that is, they still happen but they are delayed. This is adjustable to tens of thousands of transactions per second.

Name	Default	Adjustable	Description
ListScheduleGroups request rate	Each supported Region: 10	Yes	Maximum ListScheduleGroups requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval.
ListSchedules request rate	Each supported Region: 50	Yes	Maximum ListSchedules requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval.
ListTagsForResource request rate	Each supported Region: 10	Yes	Lists the tags associated with the Scheduler resource.
Number of schedule groups	Each supported Region: 500	Yes	Maximum number of schedule groups per region.

Name	Default	Adjustable	Description
Number of schedules	Each supported Region: 10,000,000	Yes	The maximum number of schedules per region. This quota includes one-time schedules that have completed running. We recommend configuring your schedules to automatically delete after completion using the <code>ActionAfterCompletion</code> feature. This is adjustable to billions of schedules.
TagResource request rate	Each supported Region: 1	Yes	Assigns one or more tags (key-value pairs) to the specified Scheduler resource.
UntagResource request rate	Each supported Region: 1	Yes	Removes one or more tags from the specified Scheduler resource.

Name	Default	Adjustable	Description
UpdateSchedule request rate	us-east-1: 1,000 us-east-2: 1,000 us-west-2: 1,000 ap-northeast-1: 1,000 ap-south-1: 1,000 ap-southeast-1: 1,000 ap-southeast-2: 1,000 eu-central-1: 1,000 eu-west-1: 1,000 eu-west-2: 1,000 sa-east-1: 1,000 Each of the other supported Regions: 250	Yes	Maximum UpdateSchedule requests per second. When you reach this quota, EventBridge Scheduler rejects requests for this operation for the remainder of the interval. This is adjustable to thousands of requests per second.

For more information about quotas and service endpoints for EventBridge Scheduler, see [Amazon EventBridge Scheduler endpoints and quotas](#) in the *AWS General Reference* guide.

Troubleshooting quotas in EventBridge Scheduler

Use the following information to help you diagnose and fix common issues that you might encounter concerning EventBridge Scheduler quotas.

ServiceQuotaExceededException

I am receiving throttling errors on `CreateSchedule`, `DeleteSchedule`, `GetSchedule`, or `UpdateSchedule` request rate, even though I am below the default rate limit.

Common cause

On September 7, 2023, EventBridge Scheduler began supporting the `ScheduleGroup` ARN (Amazon Resource Name) instead of the `Schedule` ARN in execution role trust policies. Customers allowlisted to continue using `Schedule` ARNs in their trust policy may have limits of 50 TPS, instead of the default limits of 250 to 1000 TPS (dependent on region).

Resolution

Contact [support](#) to request a higher maximum limit.

Prevention

Modify your existing trust policies in one of the following way:

- Removing all scoping from the role.
- Scoping the role so that it may be assumed using the `Schedule` ARN or the `ScheduleGroup` ARN.

For example, suppose you had the following existing trust policy:

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "scheduler.amazonaws.com"
  },
  "Action": "sts:AssumeRole",
  "Condition": {
    "StringEquals": {
      "aws:SourceArn":
        "arn:aws:scheduler:region:account:schedule/schedule_group/schedule"
    }
  }
}
```

You could update the trust policy to the following:

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "scheduler.amazonaws.com"
  },
  "Action": "sts:AssumeRole",
  "Condition": {
    "ForAnyValue:StringEquals": {
      "aws:SourceArn": [
        "arn:aws:scheduler:region:account:schedule/schedule_group/schedule",
        "arn:aws:scheduler:region:account:schedule-group/schedule_group"
      ]
    }
  }
}
```


Document history for the EventBridge Scheduler User Guide

The following table describes the documentation releases for EventBridge Scheduler.

Change	Description	Date
Changes to execution role and confused deputy prevention	This update describes changes to how the execution role is applied to a schedule group resource when you implement confused deputy prevention in the role's permission policy. • the section called “Confused deputy prevention”	September 7, 2023
Automatic deletion of schedules after completion	EventBridge Scheduler supports automatic deletion. When you configure automatic deletion, EventBridge Scheduler deletes your schedule after it's last planned invocation. • the section called “Deletion after schedule completion”	August 2, 2023
Updated topic on using universal targets	Updated the list of supported services that EventBridge Scheduler can target and integrate with. This update also includes a list of unsupported GET API operations, and includes improvements to the	March 17, 2023

universal target examples, as well as other minor improvements to across the guide.

- [the section called “Using universal targets”](#)

[Updated information on rate-based schedules that do not have a start date](#)

Added information on how EventBridge Scheduler handles rate-based schedules if you do not specify a [StartDate](#) .

March 17, 2023

- [the section called “Rate-based schedules”](#)

[New topic on managing scheduler groups](#)

Added new chapter on how to create scheduler groups with EventBridge Scheduler . Use this chapter to learn how to create a group, add schedules to the group, apply tags to more easily manage and monirot your EventBridge Scheduler resources, and finally delete a group.

March 17, 2023

- [Managing a schedule group](#)

[New topics on daylight savings time and time zones](#)

Added new sections that describe how EventBridge Scheduler handles daylight savings time, and how you can create schedules in different time zones.

November 17, 2022

- [the section called "Daylight savings time"](#)
- [the section called "Time zones"](#)

[New topic on metrics](#)

Added new topic that describes the metrics that EventBridge Scheduler publishes to CloudWatch. You can use these metrics to monitor invocation failures and understand how to resolve issues with your schedules.

November 15, 2022

- [the section called "Monitoring with CloudWatch"](#)

[Initial release](#)

Initial release of the EventBridge Scheduler User Guide.

November 10, 2022